Project 03: 常微分方程求解器

作者： Wenchong Huang

时间： April 10, 2023

目录

第 1 章 设计手册 **1**

1.1 编译与测试说明 1

1.2 参数说明 1

1.3 方法列表 2

1.4 内置方程说明 2

1.4.1 限制性三体问题 2

1.4.2 等质量一般三体问题 3

1.4.3 稳定性方程 3

1.4.4 Van der Pol 方程 4

1.5 自定义方程说明 4

1.6 类设计与继承关系 5

第 2 章 方法实现与数值实验 **6**

2.1 经典 RK 方法 6

2.1.1 收敛性测试 6

2.1.2 稳定性测试 6

2.1.3 结论 6

2.2 ESDIRK 方法 7

2.2.1 隐式方法的计算过程 7

2.2.2 收敛性测试 7

2.2.3 稳定性测试 8

2.2.4 结论 8

2.3 Gauss-Legendre 方法 8

2.3.1 方法实现与 L 稳定性分析 8

2.3.2 收敛性测试 9

2.3.3 稳定性测试 11

2.3.4 结论 11

2.4 Radau-IIA 方法 11

2.4.1 方法介绍与实现 11

2.4.2 收敛性测试 11

2.4.3 稳定性测试 13

2.4.4 结论 13

2.5 Dormand-Prince 方法 13

2.5.1 方法介绍与实现 13

2.5.2 非刚性方程测试 13

2.5.3 刚性方程测试 14

2.5.4 结论 15

2.6 Fehlberg 方法 15

2.6.1 非刚性方程测试 15

2.6.2 刚性方程测试 16

2.6.3 结论 16

2.7 自适应 Gauss-Legendre 方法 16

2.7.1 方法简介与实现 16
2.7.2 非刚性方程测试 16
2.7.3 刚性方程测试 17
2.7.4 结论 18
2.8 自适应 ESDIRK 方法 18
2.8.1 非刚性方程测试 18
2.8.2 刚性方程测试 19
2.8.3 结论 19
2.9 自适应 Radau-IIA 方法 19
2.9.1 非刚性方程测试 19
2.9.2 刚性方程测试 20
2.9.3 结论 21
2.10 对比测试：隐式自适应方法的刚性稳定性 21
2.11 Adams-Bashforth 方法 22
2.11.1 初值的敏感性及稳定的计算方法 22
2.11.2 收敛性测试 22
2.11.3 结论 24
2.12 Adams-Moulton 方法 24
2.12.1 隐式 LMM 的计算方法 24
2.12.2 收敛性测试 24
2.12.3 结论 26
2.13 BDF 方法 27
2.13.1 收敛性测试 27
2.13.2 结论 28
2.14 对比测试：所有方法的求解效果与求解效率 29

第 3 章 总结 **30**

参考文献 **31**

第 1 章 设计手册

1.1 编译与测试说明

在根目录下执行

```
make story
```

将自动编译，并生成图片与实验报告。若您想复现报告中的所有数值实验结果，请运行：

```
make run
```

1.2 参数说明

我们采用 json 文件输入，下面是一个例子

```
{
  "Problem": "Restricted 3-Body Problem",
  "Method": "Adaptive Radau-IIA",
  "Order": 5,
  "Mass Ratio": 0.012277471,
  "Init": [0.7, 0, 0, 0, -0.403526, 0],
  "T": 12.582604742222023,
  "Periodic": 1,
  "eps": 1e-8,
  "Max Step": 1,
  "Richardson": false,
  "Richardson Error Estimate": true,
  "Output": true,
  "Dense Output": false,
  "Dense-Discrete Output": true,
  "Dense-Discrete Output Step": 0.002
}
```

各参数含义如下：

- `Problem`：选择内置方程，可选的方程见 1.4 节。
- `Method`：选择求解方法，可选的方法见 1.3 节。
- `Order`：求解方法的参数（阶数或阶段数，具体见 1.3 节）。
- `Mass Ratio`：限制性三体问题的参数，表示两个大天体的质量之比。
- `Init`：初值。
- `T`：求解区间 $[0, T]$ 中的参数 T。
- `Periodic`：周期个数（若为周期解请提供该参数，否则请勿提供）。
- `eps`：精度的 Tolerance，**自适应方法必须提供该参数，固定步长方法不得提供该参数**。
- `Grid Size`：求解结点数 n，步长取为 T/n，**固定步长方法必须提供该参数，自适应方法不得提供该参数**。
- `Max Step`：自适应方法的最大步长控制值，可以不提供，默认为 $T/100$。
- `Richardson Error Estimate`：是否使用 Richardson 外插法作误差估计（开启后将进行额外的 $2n$ 步求解）。
- `Convergence Analysis`：是否估计收敛阶（对于周期解，开启后将进行额外的 $2n$ 步求解；对于非周期解，需要与上一个参数同时开启，将进行额外的 $2n$ 与 $4n$ 步求解）。

- `Output`：是否输出求解结点上的计算结果。
- `Dense Output`：是否输出每一段区间的插值多项式。
- `Dense Discrete Output`：是否根据插值多项式等距离取点输出。
- `Dense Discrete Output Step`：根据插值多项式等距离取点输出的步长，默认为 0.01。

1.3 方法列表

我们的求解器支持的线性多步法如下表所示，表中的“方法名称”即求解器用于识别方法的字符串。

方法名称	简称	计算类型	步长类型	步数	阶数	零稳定	A-稳定
Adams-Bashforth	ABp	显式	固定	p	p	是	
Adams-Moulton	AMp $(p \geq 2)$	隐式	固定	$p-1$	p	是	$p \leq 2$ 时
BDF	BDFp	隐式	固定	p	p	$p \leq 6$ 时	$p \leq 2$ 时

支持的 RK 方法如下表所示，表中的“方法名称”即求解器用于识别方法的字符串。

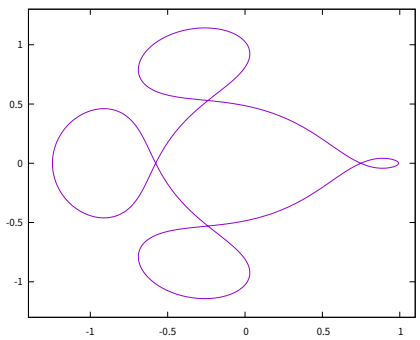
方法名称	简称	计算类型	步长类型	阶段数	阶数	A-稳	B-稳	L-稳
Classical RK	CLRK	显式	固定	4	4			
ESDIRK	ESDIRK	隐式	固定	6	4	是	是	是
Gauss-Legendre	GLRKs	隐式	固定	s	$2s$	是	是	
Radau-IIA	RIIARKs	隐式	固定	s	$2s-1$	是	是	是
Fehlberg	FB4(5)	显式	自适应	6	4			
Dormand-Prince	DOPRI5	显式	自适应	7	5			
Dormand-Prince 8(7)	DOPRI8	显式	自适应	13	8			
Adaptive ESDIRK	A-ESDIRK	隐式	自适应	6	4	是	是	是
Adaptive Gauss-Legendre	A-GLRKs	隐式	自适应	s	$2s$	是	是	
Adaptive Radau-IIA	A-RIIARKs	隐式	自适应	s	$2s-1$	是	是	是

凡是带阶数参数的方法，我们都没有内置系数表，而是通过公式计算系数，因此我们不对 s 或 p 的最大取值做限制。但是用户将阶数设置得过高时，实际效果可能反而不如较低阶的方法。

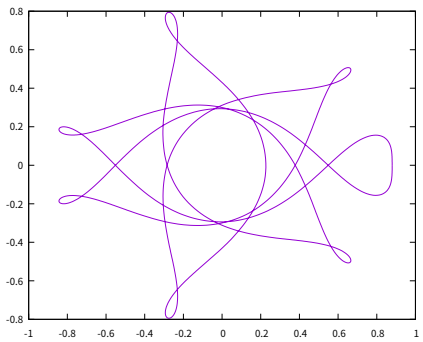
1.4 内置方程说明

1.4.1 限制性三体问题

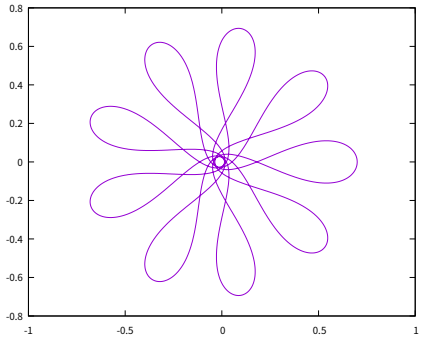
这是讲义 11.7 节的测试方程，将`"Problem"`参数设为`"Restricted 3-Body Problem"`即可直接使用。你需要提供参数`"Mass Ratio"`，表示两个大天体的质量之比。在`samples`文件夹中，我们提供了四个测试轨道，命名为`orbits1-4.json`，四个轨道如下。



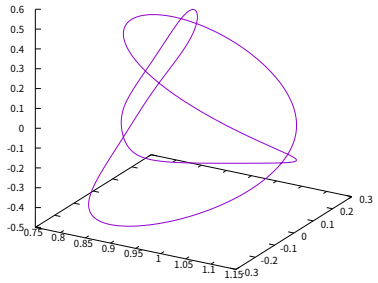
轨道一



轨道二



轨道三



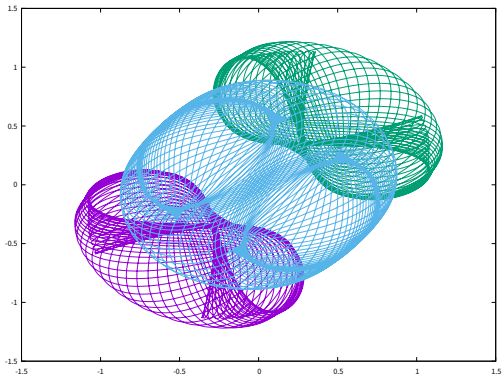
轨道四

轨道一来自 Hairer 的经典教材 [4]，可以维持至少 5 个周期，需要较高阶的计算方法，例如我们提供的 5 阶段 9 阶 A-RIIARK 方法。轨道二可以维持至少 5 个周期，并不需要非常高阶的方法。轨道三因为距离月球较远，退化成了绕地卫星问题，因此轨道相对稳定，只要精度不要太差，大部分方法都能计算至少 15 个周期。轨道四的数据来自 NASA 官网[1]。

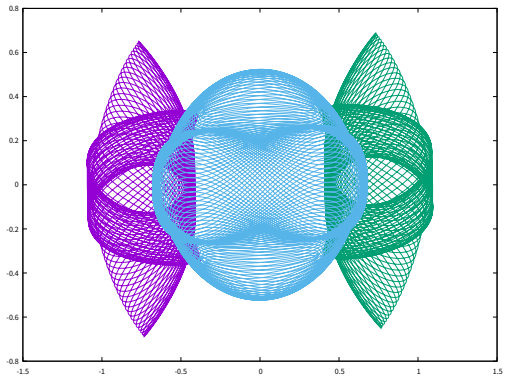
我们生成了这些轨道的动态视频，您可以在线观看[2]。**我们将以轨道一与轨道二作为样本，测试各方法的收敛阶数，并通过轨道一的测试观察自适应方法的显著优势。**

1.4.2 等质量一般三体问题

这是三颗等质量恒星在互相的引力作用下运动的方程，将`"Problem"`参数设为`"3-Body Problem"`即可直接使用。初值共有 18 个变量，依次表示三个恒星的空间坐标、三个恒星的初速度。在`samples`文件夹中，我们提供了三个测试轨道，命名为`orbits5-7.json`，其中两个轨道如下。



轨道五



轨道六

这两个轨道由李晓明等人于 2017 年发现 [9]。我们生成了这些轨道的动态视频，您可以在线观看[2]。该方程仅供观赏，不参与数值实验中的测试。

1.4.3 稳定性方程

这个方程是经典的稳定性方程：

$$u' = \lambda u, \quad u(0) = 1. \tag{1.1}$$

其精确解为 $u(t) = e^{\lambda t}$，我们的内置参数为 $\lambda = -10^6$，将`"Problem"`参数设为`"Stability Problem"`即可直接使用。这个方程可以体现数值方法的稳定性：

[1] https://ssd.jpl.nasa.gov/tools/periodic_orbits.html

[2] https://www.bilibili.com/video/BV1GM4y1t7av/

- 不稳定方法：除非步长极小，否则仅需几步，数值解就发散到 ∞。
- A 稳定方法：不论步长为多少，数值解始终有界，模长递减收敛至 0，收敛速度不能保证，可能高频振荡。
- L 稳定方法：具有 A 稳定方法的表现，且迅速收敛至 0，高频振荡将被抑制。

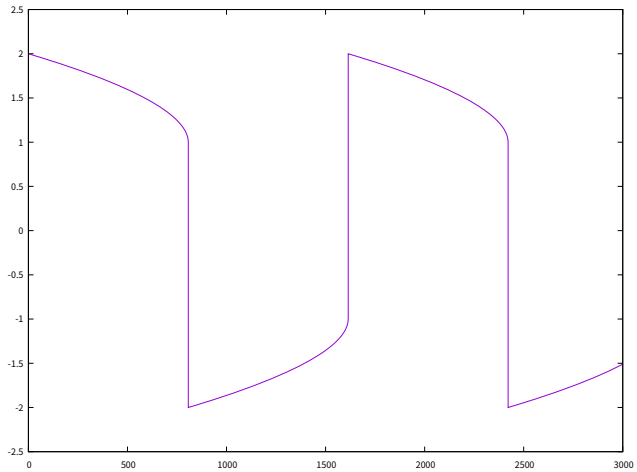
在数值实验中，我们将使用这个方程验证各方法的稳定性。

1.4.4 Van der Pol 方程

这个方程的表达式如下：

$$u'' = \lambda(1-u^2)u' - u, \quad u(0) = 2, \quad u'(0) = 0. \tag{1.2}$$

它没有解析解。我们的内置参数为 $\lambda = 1000$，将`"Problem"`参数设为`"Van der Pol Problem"`即可直接使用。方程的数值解如下图所示。



它是一个典型的刚性方程，使用固定步长方法几乎无法求解，除非步长极小。自适应步长的求解方法也并非都适用于该方程，采用显式方法需要的步数非常多，求解速度很慢，只有自适应步长的隐式方法才能快速求解这个方程。

在数值实验中，我们将使用这个方程比较自适应步长的显式方法与隐式方法。

1.5 自定义方程说明

如果您想要自定义方程，请编写自己的代码，在头文件中包含`"IVP.h"`，您的方程函数需要继承下述基类。

```
class TimeFunction{
  public:
     virtual ColVector operator () (const ColVector &x, const double &t) const = 0;
  };
```

您需要完成虚函数`operator ()`，这表示您的方程 $\mathbf{u}'(\mathbf{x},t) = \mathbf{f}(\mathbf{x},t)$ 中的函数 $\mathbf{f}$。

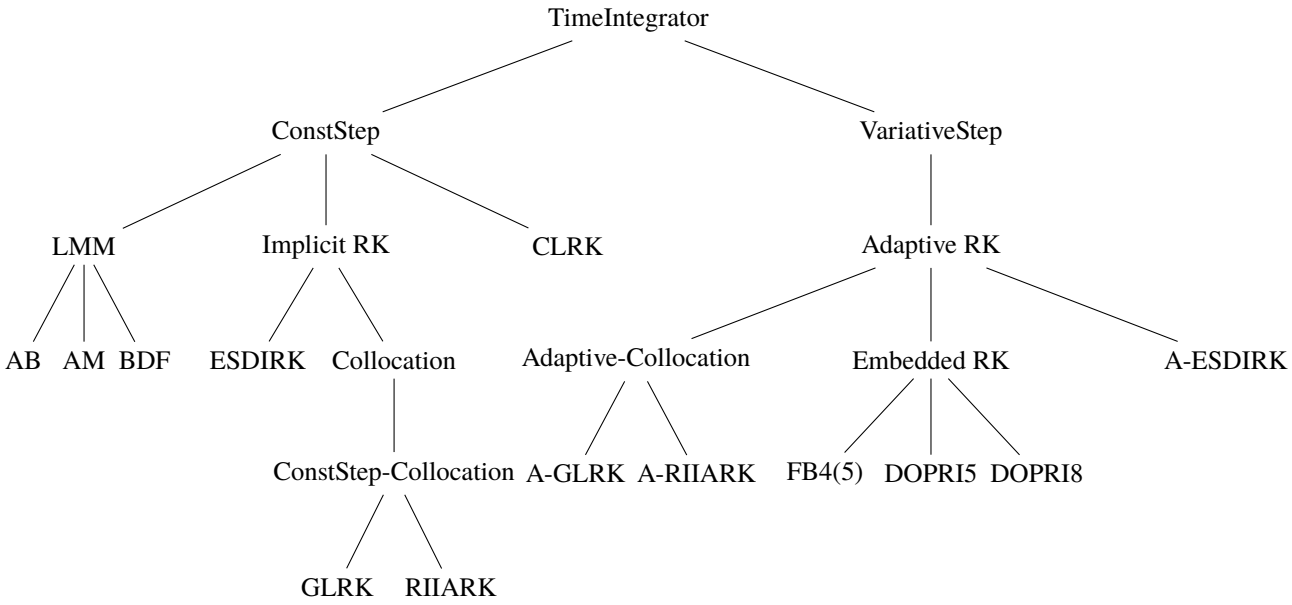
您可以使用下面的代码创建求解器并求解方程。

```
auto& factory = TimeIntegratorFactory::Instance();
auto solver = factory.createTimeIntegrator("Radau-IIA", 4); //替换成您需要的方法
solver->solveWithInfo(*f, x0, T, GridSize, true);   //true表示进行误差估计
solver->output("result.txt");                       //将结果输出到文件
solver->denseOutput("result-dense.txt");            //将结果的插值多项式输出到文件
```

更多用法请参考`solve.cpp`。

1.6 类设计与继承关系

我们以TimeIntegrator为基类，派生出了各种方法。为了充分复用代码，我们设计了一系列继承关系：



上面的树状图中，有一个多继承关系没有画出：因为除了步长控制，ConstStep-Collocation与Adaptive-Collocation的一步求解是一样的，因此共同继承了一个 Collocation 方法的一步求解器。注意，上面的图仅表继承关系，名称并不是代码中类的名称，但大致相同。

对于求解接口，基类提供固定步长与自适应步长两种接口，定义为虚函数。在固定步长求解器基类中禁用自适应步长求解方法，在自适应步长求解器基类中禁用固定步长求解方法。

IVP中内置了工厂TimeIntegratorFactory，这是一个单例类，设计如下：

```
class TimeIntegratorFactory{
  public:
      using CreateTimeIntegratorCallBack = TimeIntegrator* (*)(int);
  private:
      using CallbackMap = std::map<std::string, CreateTimeIntegratorCallBack>;
  public:
      bool registerTimeIntegrator(const std::string &ID, CreateTimeIntegratorCallBack createFn);
      bool unregisterTimeIntegrator(const std::string &ID);
      TimeIntegrator* createTimeIntegrator(const std::string &ID, const int &order);
  private:
      CallbackMap callbacks_;
  private:
      TimeIntegratorFactory() = default;
      TimeIntegratorFactory(const TimeIntegratorFactory&) = default;
      TimeIntegratorFactory& operator = (const TimeIntegratorFactory&) = default;
      ~TimeIntegratorFactory() = default;
  public:
      static TimeIntegratorFactory& Instance();
};
```

求解器的注册利用__attribute__((constructor))属性，在主函数开始之前执行。用户只需包含头文件IVP.h，使用工厂方法createTimeIntegrator创建求解器即可。

第 2 章 方法实现与数值实验

2.1 经典 RK 方法

2.1.1 收敛性测试

在本文中，我们均以限制性三体问题作为收敛性测试的例子。方程与参数、初值见讲义 11.7 节。

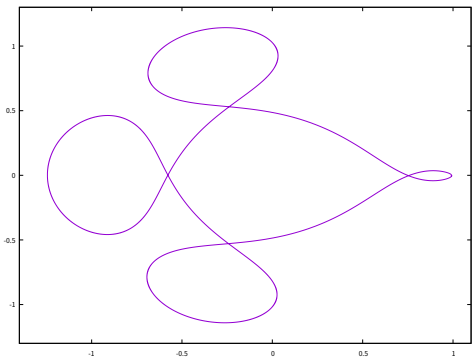
使用经典 RK 方法，令步长为 $\frac{T}{n}$，取不同的 n，将 T 时刻的数值解与初始点比较误差。对轨道一测试如下。

n $(\times 10^5)$		1	2	4	8	收敛阶
T 时刻误差	u_1	1.04×10^{-6}	6.32×10^{-8}	3.89×10^{-9}	2.40×10^{-10}	4.02
	u_2	3.27×10^{-6}	1.98×10^{-7}	1.22×10^{-8}	7.51×10^{-10}	4.02
	u_4	5.33×10^{-4}	3.23×10^{-5}	1.99×10^{-6}	1.22×10^{-7}	4.02
	u_5	1.62×10^{-4}	9.83×10^{-6}	6.06×10^{-7}	3.74×10^{-8}	4.02
求解时间 (s)		0.158	0.327	0.597	1.22	

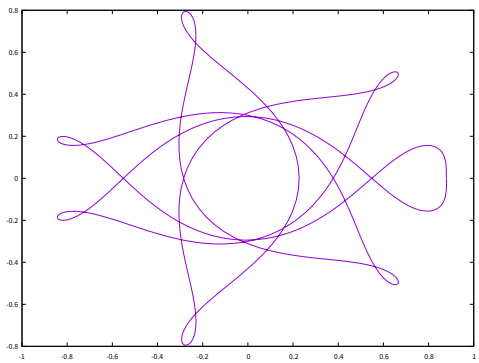
对轨道二测试如下，这里的收敛阶用 Richardson 外插法估计。

n		450	900	1800	收敛阶
与 $2n$ 步的解在 T 时刻的误差	u_1	0.0173229	0.000558742	1.57643e-05	5.15
	u_2	0.0196696	0.000638166	2.07028e-05	4.94
	u_4	0.0670598	0.00236742	6.64987e-05	5.16
	u_5	0.0386287	0.00114744	3.2626e-05	5.14
求解时间 (s)		0.00199	0.00128	0.00302	

要想绘制出仅用肉眼无法辨别的图像，轨道一需要取 $n = 15000$，轨道二需要取 $n = 800$，绘制的图像如下。



轨道一，$n = 15000$



轨道二，$n = 800$

2.1.2 稳定性测试

使用稳定性方程 (1.1) 测试经典 RK 方法，求解区间为 $[0, 1]$，步长为 $\frac{1}{n}$，结果如下表。当数值解为-nan 时我们称为“溢出”，当数值解的绝对值首次小于 10^{-5} 时，我们称为“收敛”。

n	3.5×10^5	3.6×10^5	3.7×10^5
T 时刻的数值解	-nan	1.97×10^{-306}	1.86×10^{-307}
溢出或收敛所用步数	6445	1016	93
溢出或收敛的时刻	0.01841	0.002822	0.000251

2.1.3 结论

收敛性测试的结果表明，经典 RK 方法的累积误差约 $O(k^4)$，是一个四阶精度的方法。

稳定性测试的结果表明，经典 RK 方法不具有 A 稳定性，且绝对稳定区域较小，因此不适用于刚性方程。

2.2 ESDIRK 方法

事实上，ESDIRK 是一族方法，我们仅实现了其中一个，该方法的 Butcher 表在讲义中已经给出。我们这里指出，讲义中给出的这个 ESDIRK 方法具有 L 稳定性，见参考文献 [5]。

2.2.1 隐式方法的计算过程

考虑到 ESDIRK 是隐式方法，我们需要解四个非线性方程，每个方程形如：

$$\mathbf{y}_i = \mathbf{f}(\mathbf{a} + c\mathbf{y}_i, b). \tag{2.1}$$

其中 $\mathbf{a}, b, c$ 是可以事先计算好的常量，$\mathbf{y}_i$ 是 m 维向量，m 为微分方程的变量个数。对于这样的方程，我们给出**第一种计算方法**：

1. $\mathbf{y}_i \leftarrow \mathbf{0}$.
2. $\hat{\mathbf{y}}_i \leftarrow \mathbf{y}_i$, $\mathbf{y}_i \leftarrow \mathbf{f}(\mathbf{a} + c\mathbf{y}_i, b)$.
3. 若 $||\hat{\mathbf{y}}_i - \mathbf{y}_i|| < \varepsilon$，结束，否则返回步骤 2.

实践告诉我们，上述计算过程在很多刚性方程中并不能收敛。因此我们还要给出**第二种方法**：

1. 设 $\mathbf{g}(\mathbf{x}) = \mathbf{f}(\mathbf{a} + c\mathbf{x}, b) - \mathbf{x}$.
2. 以 $\mathbf{0}$ 为初值，使用牛顿迭代法求 $\mathbf{g}(\mathbf{x})$ 的零点.

上面两种方法中，我们优先使用第一种，当第一种方法迭代 50 次而无法收敛时，再采取第二种方法，若还不收敛，则求解失败。

事实上，我们还面临一个巨大的问题——**数值微分的不稳定性**。我们采用第二种方法时，牛顿迭代需要求 Jacobi 矩阵，并对其做 LU 分解，这对微分的精度有较高的要求。要求用户提供导函数的表达式是不现实的，限制性三体问题的求导对人类来说过于复杂。Jorge Nocedal 在他的著作 [6] 中给出了一种解析求导的方法，基本思路为将函数分解为一系列初等函数进行求导，但这需要用户提供计算路线图。

在我们的求解器中，仍然采用数值微分格式：

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}. \tag{2.2}$$

确定 h 的取值非常困难，起初我们为了精度，将 h 取得很小，然而这带来了严重的数值不稳定性，这导致牛顿迭代难以收敛。在王何宇老师的建议下，我们根据参考资料 [10]，确定了 h 的最优取值为

$$h = \sqrt[3]{\frac{3\epsilon}{M}}. \tag{2.3}$$

其中 ϵ 表示机器精度，$M = \sup|f'''(x)|$。在我们的求解器中，h 被估计为 3×10^{-6}。

另一个问题是——**Jacobi 矩阵奇异**。在某些点处可能会发生这个情况，导致牛顿迭代失败。为此，可以使用 Gill-Murray 修正牛顿法 [2] 或其它方法。但由于我们并不是做专门的优化求解器，因此我们只做粗略修正：

$$J \leftarrow J + 0.01I. \tag{2.4}$$

经过测试，用上述步骤计算隐式方法的迭代格式，在数值上较为稳定，在四个内置方程中均表现良好。

2.2.2 收敛性测试

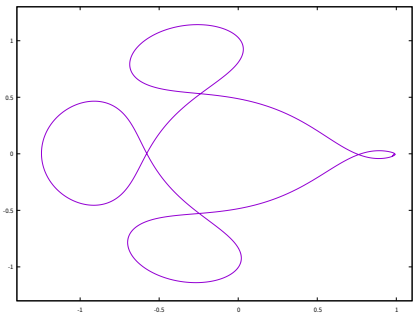
使用 ESDIRK 方法，令步长为 $\frac{T}{n}$，取不同的 n，将 T 时刻的数值解与初始点比较误差。对轨道一测试如下。

n ($\times 10^5$)		0.5	1	2	4	收敛阶
T 时刻误差	u_1	5.78×10^{-6}	3.62×10^{-7}	2.27×10^{-8}	1.42×10^{-9}	4.00
	u_2	1.88×10^{-5}	1.18×10^{-6}	7.39×10^{-8}	4.63×10^{-9}	4.00
	u_4	3.06×10^{-3}	1.92×10^{-4}	1.20×10^{-5}	7.54×10^{-7}	4.00
	u_5	8.94×10^{-4}	5.64×10^{-5}	3.53×10^{-6}	2.22×10^{-7}	4.00
求解时间 (s)		0.519	1.06	2.18	4.41	

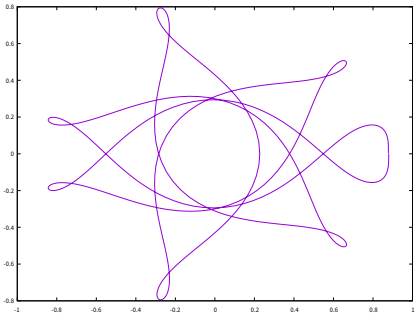
对轨道二测试如下，这里的收敛阶用 Richardson 外插法估计。

n		500	1000	2000	收敛阶
与 $2n$ 步的解在 T 时刻的误差	u_1	0.000844317	3.19044e-05	1.16815e-06	4.77
	u_2	0.000722844	2.50061e-05	7.46045e-07	5.07
	u_4	0.00365792	0.000138021	5.08278e-06	4.76
	u_5	0.00167582	6.32808e-05	2.29493e-06	4.79
求解时间 (s)		0.0144	0.0196	0.0342	

要想绘制出仅用肉眼无法辨别的图像，轨道一需要取 $n = 9000$，轨道二需要取 $n = 700$，绘制的图像如下。



轨道一，$n = 9000$



轨道二，$n = 700$

与经典 RK 方法相比，虽然 ESDIRK 的计算相对准确，因此点数取得少一些，但是求解时间却更长。

2.2.3 稳定性测试

使用稳定性方程 (1.1) 测试 ESDIRK 方法，求解区间为 $[0, 1]$，步长为 $\frac{1}{n}$，结果如下表。当数值解为-nan 时我们称为“溢出”，当数值解的绝对值首次小于 10^{-5} 时，我们称为“收敛”。

n	10	100	1000	10000
T 时刻的数值解	1.19×10^{-24}	6.87×10^{-24}	8.00×10^{-21}	2.21×10^{-20}
溢出或收敛所用步数	2	2	3	6
溢出或收敛的时刻	0.2	0.02	0.003	0.0005

注意，这里 T 时刻的数值解不具备比较意义，因为隐式方法的求解基于迭代，而当解的绝对误差达到机器精度时，迭代会立刻停止，并不会像显式方法一样继续下降到 10^{-308} 级别。

事实上，对于讲义上的刚性方程 (11.95)，取 $\lambda = -10^6$ 时，不论取步长为多少，ESDIRK 方法总能得到稳定解。这个测试我们不在本文展示。

2.2.4 结论

收敛性测试的结果表明，ESDIRK 方法的一步误差约 $O(k^5)$，累积误差约 $O(k^4)$，是一个四阶精度的方法。

稳定性测试的结果表明，ESDIRK 方法具有 A 稳定性、L 稳定性，具有一定的求解刚性方程的能力。改造成自适应方法后，可能会适用于刚性方程（见 A-ESDIRK 的测试）。

2.3 Gauss-Legendre 方法

2.3.1 方法实现与 L 稳定性分析

在 GLRK 方法的实现中，我们并没有直接使用讲义上的 Butcher 表，而是使用数值代数的方法求多项式 (2.5) 的根，再由讲义上的 (11.150) 计算 A 与 $\mathbf{b}$，这样我们就可以支持任意的 s。例如当 $s = 5$ 时，我们生成的 Butcher 表如下。

0.0469101	0.0592317	-0.0195704	0.0112544	-0.00559379	0.00158811
0.230765	0.128151	0.119657	-0.0245921	0.0103183	-0.00276899
0.5	0.113776	0.260005	0.142222	-0.0206903	0.00468715
0.769235	0.121232	0.228996	0.309037	0.119657	-0.00968756
0.95309	0.116875	0.244908	0.27319	0.258885	0.0592317
	0.118463	0.239314	0.284444	0.239314	0.118463

GLRK 是隐式方法，且 $\mathbf{y}_i$ 与 $\mathbf{y}_1, ..., \mathbf{y}_s$ 均有关，因此不能像 ESDIRK 一样逐个迭代以求出 $\mathbf{y}_i$，而是需要一起迭代。我们给出**第一种方法**：

1. $\mathbf{y}_i \leftarrow \mathbf{f}(\mathbf{U}^n, t_n)$，$(i = 1, ..., s)$.
2. 对 $i = 1, ..., s$ 依次做：$\hat{\mathbf{y}}_i \leftarrow \mathbf{y}_i$，$\mathbf{y}_i \leftarrow \mathbf{f}(\mathbf{U}^n + k\sum_{j=1}^s a_{ij}\mathbf{y}_j, t_n + c_i k)$.
3. 若 $\max_{i=1}^s ||\hat{\mathbf{y}}_i - \mathbf{y}_i|| < \varepsilon$，结束，否则返回步骤 2.

实践告诉我们，上述计算过程在很多刚性方程中并不能收敛。因此我们还要给出**第二种方法**：牛顿迭代法。两种方法中，我们优先使用第一种，当第一种方法迭代 50 次而无法收敛时，再采取第二种方法，若还不收敛，则求解失败。

这里我们牛顿迭代法要求解的是一个 sm 元函数的零点，m 为微分方程的变量个数。每次迭代需要计算 Jacobi 矩阵并做 LU 分解，一个牛顿步的计算的代价高达 $O((sm)^3)$。在刚性方程的求解中，刚性区域的牛顿迭代占用了大部分的时间。

讲义的 Example 11.265 指出，GLRK3 不具有 L 稳定性，事实上我们有更一般的结论：

定理 2.1

对任意的 $s \geq 1$，GLRKs 均具有 A 稳定性、B 稳定性，但不具有 L 稳定性。

证明 根据 [3] 第 IV 章习题 3，一个配点法具有 L 稳定性的必要条件是存在 $c_i = 1$. GLRK 的 $c_1, ..., c_s$ 是多项式：

$$P_s(t) = \frac{(s!)^2}{(2s)!}\sum_{j=0}^{s}(-1)^{s-j}\binom{s}{j}\binom{s+j}{j}t^j. \tag{2.5}$$

的所有根。存在 $c_i = 1$ 意味着 $P_s(1) = 0$，然而：

$$P_s(1) = \frac{(s!)^2}{(2s)!}\sum_{j=0}^{s}(-1)^{s-j}\binom{s}{j}\binom{s+j}{j} = \frac{(s!)^2}{(2s)!} \neq 0.$$

矛盾，因此不存在 $c_i = 1$，所以 GLRK 不具有 L 稳定性。A 稳定性与 B 稳定性见讲义中的定理 11.283。

2.3.2 收敛性测试

使用 GLRKs 方法，令步长为 $\frac{T}{n}$，取不同的 n，对轨道一测试，将 T 时刻的数值解与初始点比较误差，当 $s = 1$ 时，即一阶段二阶 GLRK 方法，结果如下。

n ($\times 10^6$)		2.5	5	10	20	收敛阶
T 时刻误差	u_1	1.21×10^{-5}	3.02×10^{-6}	7.54×10^{-7}	1.88×10^{-7}	2.00
	u_2	3.82×10^{-5}	9.55×10^{-6}	2.39×10^{-6}	5.97×10^{-7}	2.00
	u_4	6.24×10^{-3}	1.56×10^{-3}	3.89×10^{-4}	9.72×10^{-5}	2.00
	u_5	1.86×10^{-3}	4.68×10^{-4}	1.17×10^{-4}	2.93×10^{-5}	2.00
求解时间 (s)		5.88	10.1	20.6	41.6	

$s = 2$ 结果如下。

n ($\times 10^4$)		2.5	5	10	20	收敛阶
T 时刻误差	u_1	6.79×10^{-5}	4.23×10^{-6}	2.65×10^{-7}	1.66×10^{-8}	4.00
	u_2	1.95×10^{-4}	1.24×10^{-5}	7.80×10^{-7}	4.88×10^{-8}	4.00
	u_4	3.23×10^{-2}	2.03×10^{-3}	1.27×10^{-4}	7.97×10^{-6}	4.00
	u_5	9.92×10^{-3}	6.56×10^{-4}	4.13×10^{-5}	2.58×10^{-6}	4.00
求解时间 (s)		0.219	0.349	0.567	1.48	

$s=3$ 结果如下。

n $(\times 10^4)$		0.875	1.75	3.5	7	收敛阶
T 时刻误差	u_1	1.25×10^{-4}	1.75×10^{-6}	2.70×10^{-8}	4.20×10^{-10}	6.01
	u_2	3.79×10^{-4}	5.49×10^{-6}	8.46×10^{-8}	1.32×10^{-9}	6.01
	u_4	6.35×10^{-2}	8.95×10^{-4}	1.38×10^{-5}	2.15×10^{-7}	6.01
	u_5	1.69×10^{-2}	2.72×10^{-4}	4.20×10^{-6}	6.54×10^{-8}	6.01
求解时间 (s)		0.0833	0.151	0.298	0.522	

对轨道二测试，这里的收敛阶用 Richardson 外插法估计，$s=1$ 结果如下。

n $(\times 10^4)$		1	2	4	8	收敛阶
与 $2n$ 步的解在 T 时刻的误差	u_1	0.000221967	5.55177e-05	1.3881e-05	3.47036e-06	2.00
	u_2	7.95868e-05	1.99551e-05	4.99243e-06	1.24833e-06	2.00
	u_4	0.00100695	0.000251523	6.28674e-05	1.5716e-05	2.00
	u_5	0.000404512	0.000101212	2.53082e-05	6.32738e-06	2.00
求解时间 (s)		0.0348	0.0691	0.109	0.211	

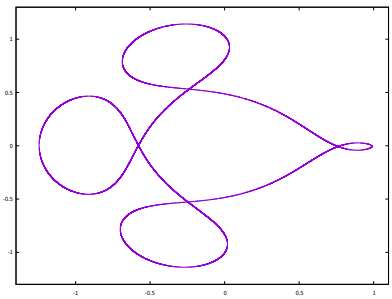
$s=2$ 结果如下。

n $(\times 10^3)$		1	2	4	8	收敛阶
与 $2n$ 步的解在 T 时刻的误差	u_1	2.63427e-05	1.65245e-06	1.03375e-07	6.46248e-09	4.00
	u_2	8.14391e-06	5.10769e-07	3.19459e-08	1.99689e-09	4.00
	u_4	0.000119076	7.46844e-06	4.67207e-07	2.92074e-08	4.00
	u_5	4.80285e-05	3.01287e-06	1.8848e-07	1.17829e-08	4.00
求解时间 (s)		0.0339	0.0563	0.0822	0.145	

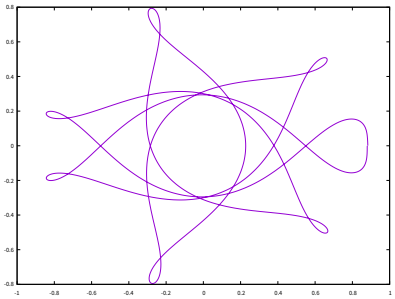
$s=3$ 结果如下。

n		450	900	1800	3600	收敛阶
与 $2n$ 步的解在 T 时刻的误差	u_1	2.9731e-06	4.75577e-08	7.47275e-10	1.18147e-11	5.99
	u_2	6.89595e-07	1.09705e-08	1.72299e-10	2.54906e-12	6.02
	u_4	1.33968e-05	2.14281e-07	3.36701e-09	5.32073e-11	5.99
	u_5	5.42078e-06	8.67106e-08	1.36245e-09	2.15747e-11	5.99
求解时间 (s)		0.0322	0.0494	0.088	0.15	

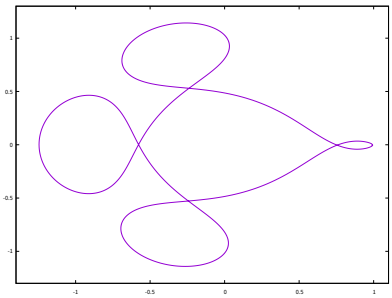
绘图结果如下，n 均取尽可能小，并保证图像能准确到看不出差异。



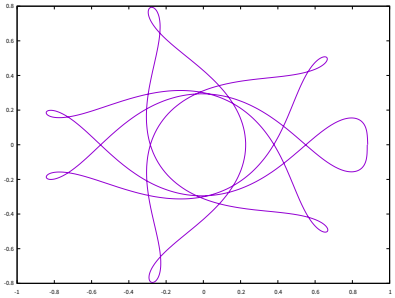
$s=1$，$n=110000$，0.358 秒



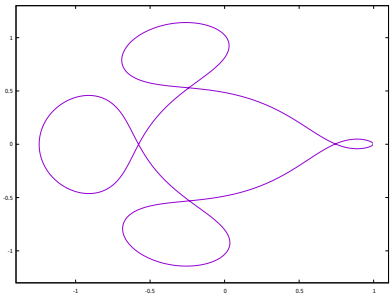
$s=1$，$n=3000$，0.0101 秒



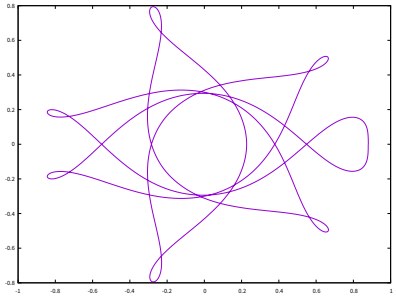
$s=2$，$n=9000$，0.0671 秒



$s=2$，$n=320$，0.0043 秒



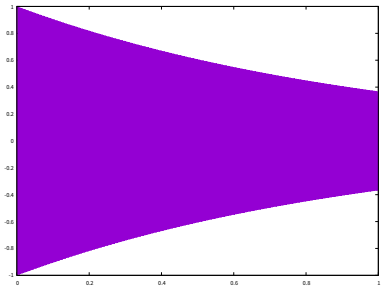
$s=3$，$n=3500$，0.0461 秒



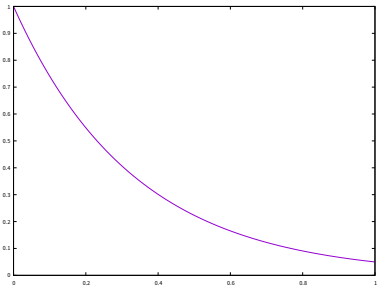
$s=3$，$n=160$，0.0096 秒

2.3.3 稳定性测试

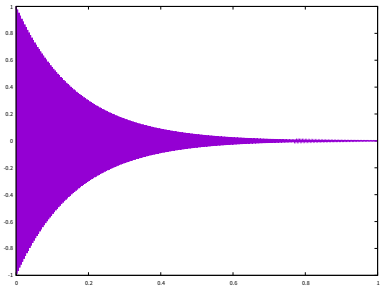
使用稳定性方程 (1.1) 测试 GLRKs 方法 ($s=1,2,3$)，求解区间为 $[0,1]$，步长为 0.002，将数值解作图如下。可以看到，数值解始终保持有界，且绝对值随时间递减。但是收敛速度极慢，且 $s=1,3$ 时高频振荡。



$s=1$



$s=2$



$s=3$

2.3.4 结论

收敛性测试的结果表明，GLRKs 方法的一步误差约 $O(k^{2s+1})$，累积误差约 $O(k^{2s})$，是一个 $2s$ 阶精度的方法。用户可以将 s 设置得任意大，但不建议超过 5。

稳定性测试的结果表明，GLRKs 方法具有 A 稳定性，不具有 L 稳定性，求解刚性方程的能力较弱。

2.4 Radau-IIA 方法

2.4.1 方法介绍与实现

Radau-IIA 方法是一种具有 L 稳定性的配点法 [1]，事实上，讲义中的 Example 11.267 就是 3 阶段 5 阶 Radau-IIA 方法。下面我们来构造一般的 s 阶段 $2s-1$ 阶 Radau-IIA 方法。我们设多项式

$$P_s(x)=\frac{\mathrm{d}^{s-1}}{\mathrm{d}x^{s-1}}x^{s-1}(x-1)^s. \tag{2.6}$$

Radau-IIA 方法的 $c_1,\ldots,c_s$ 是 $P_s(x)$ 的所有根（从小到大排列），再由讲义上的 (11.150) 计算 A 与 $\mathbf{b}$。同样的，我们不内置 Butcher 表，而是通过数值代数的方法求根，从而可以支持任意大的 s。

通过上述方法构造好 Butcher 表后，求解过程与 GLRK 完全一样。在代码实现时，求解过程被定义在它们共同的父类中。

2.4.2 收敛性测试

使用 RIIARKs 方法，令步长为 $\frac{T}{n}$，取不同的 n，对轨道一测试，将 T 时刻的数值解与初始点比较误差，当 $s=2$ 时，即二阶段三阶 Radau-IIA RK 方法，结果如下。

n ($\times 10^5$)		1	2	4	收敛阶
T 时刻误差	u_1	7.52×10^{-5}	9.12×10^{-6}	1.13×10^{-6}	3.01
	u_2	2.34×10^{-4}	2.92×10^{-5}	3.64×10^{-6}	3.00
	u_4	3.88×10^{-2}	4.76×10^{-3}	5.93×10^{-4}	3.01
	u_5	1.07×10^{-2}	1.40×10^{-3}	1.76×10^{-4}	2.99
求解时间 (s)		0.511	1.35	3.55	

$s = 3$ 结果如下。

n ($\times 10^4$)		4	8	16	收敛阶
T 时刻误差	u_1	3.23×10^{-7}	1.00×10^{-8}	3.09×10^{-10}	5.02
	u_2	1.11×10^{-6}	3.43×10^{-8}	1.06×10^{-9}	5.01
	u_4	1.80×10^{-4}	5.58×10^{-6}	1.73×10^{-7}	5.01
	u_5	5.03×10^{-5}	1.56×10^{-6}	4.81×10^{-8}	5.02
求解时间 (s)		0.475	0.755	2.41	

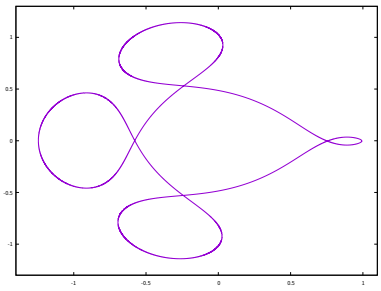
对轨道二测试，这里的收敛阶用 Richardson 外插法估计，$s = 2$ 结果如下。

n ($\times 10^4$)		2	4	8	收敛阶
一步误差	u_1	3.96162e-06	4.95246e-07	6.19073e-08	3.00
	u_2	3.98663e-06	4.98329e-07	6.22902e-08	3.00
	u_4	1.68681e-05	2.10876e-06	2.63602e-07	3.00
	u_5	8.08741e-06	1.01098e-06	1.26375e-07	3.00
求解时间 (s)		0.176	0.332	0.581	

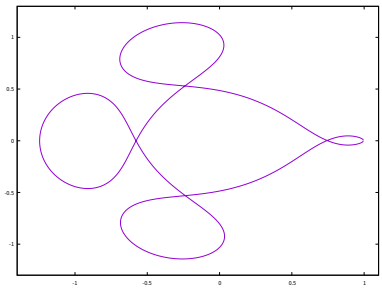
$s = 3$ 结果如下。

n ($\times 10^3$)		2	4	8	收敛阶
一步误差	u_1	8.75807e-07	2.74218e-08	8.57287e-10	5.00
	u_2	8.80329e-07	2.75448e-08	8.60868e-10	5.00
	u_4	3.73978e-06	1.17098e-07	3.6609e-09	5.00
	u_5	1.77721e-06	5.56413e-08	1.73946e-09	5.00
求解时间 (s)		0.0304	0.0858	0.242	

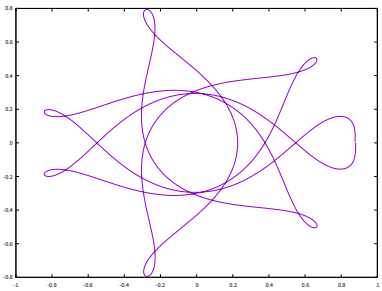
绘图结果如下，n 均取尽可能小，并保证图像能准确到看不出差异。



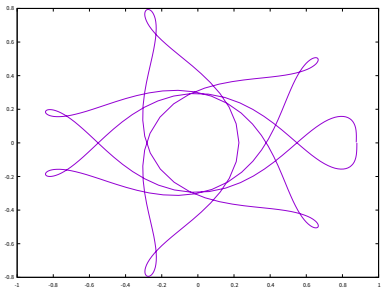
$s = 2$，$n = 30000$，0.157 秒



$s = 3$，$n = 4500$，0.0527 秒



$s = 2$，$n = 2200$，0.0573 秒



$s = 3$，$n = 380$，0.0096 秒

2.4.3 稳定性测试

使用稳定性方程 (1.1) 测试 Radau-IIA 方法，求解区间为 $[0,1]$，步长为 0.02，结果如下表。当数值解的绝对值首次小于 10^{-5} 时，我们称为“收敛”。

s	2	3	4
收敛所用步数	2	2	2
收敛的时刻	0.04	0.04	0.04

2.4.4 结论

收敛性测试的结果表明，s 阶段 Radau-IIA 方法的一步误差约 $O(k^{2s})$，累积误差约 $O(k^{2s-1})$，是一个 $2s-1$ 阶精度的方法。用户可以将 s 设置得任意大，但不建议超过 5。

稳定性测试的结果表明，Radau-IIA 方法具有 A 稳定性、L 稳定性，有求解刚性方程的能力。

2.5 Dormand-Prince 方法

2.5.1 方法介绍与实现

我们实现了 DOPRI5 与 DOPRI8 两种 Dormand-Prince 方法，前者是讲义中的方法，后者是 matlab 的求解器`ode87`采用的方法，是最为流行的高阶数值积分方法。DOPRI8 的构造方法与 Butcher 表见 Prince 和 Dormand 的文章 [7]。

步长控制采用了讲义 Formula 11.235 中的方法，它被写在了所有自适应方法的共同基类中。

另外，由于使用 Dormand-Prince 方法求解非刚性方程时，所需的结点通常很少，这会使我们画出的图看上去不够光滑，因此我们用 3 阶 2 次 Hermite 插值填充结点之间的值，并以 0.01 的时间间隔输出。

2.5.2 非刚性方程测试

使用限制性三体问题作为样例测试 DOPRI5 方法。Tolerance 从 3.2768×10^{-9} 开始，每次减小至原来的 $\frac{1}{32}$，观察 DOPRI5 的步数需要增加至多少。若步数的增长率接近 2，则可以验证 DOPRI5 方法具有 5 阶收敛性。轨道一的结果如下表。

Tolerance($\times10^{-13}$)	32768	1024	32	1	增长率
所需步数	472	944	1887	3775	2.00
计算时间 (s)	0.00398	0.00715	0.0128	0.0246	

轨道二的结果如下表。

Tolerance($\times10^{-13}$)	32768	1024	32	1	增长率
所需步数	739	1477	2953	5906	2.00
计算时间 (s)	0.00563	0.00979	0.0218	0.0423	

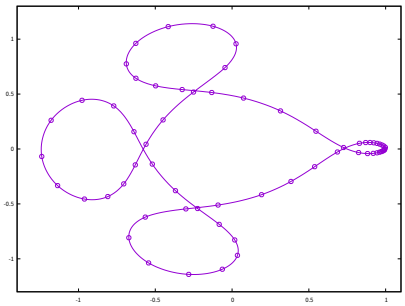
同样地，我们测试 DOPRI8 方法。Tolerance 从 6.5536×10^{-9} 开始，每次减小至原来的 $\frac{1}{256}$，观察 DOPRI8 的步数需要增加至多少。若步数的增长率接近 2，则可以验证 DOPRI8 方法具有 8 阶收敛性。轨道一结果如下表。

Tolerance($\times10^{-13}$)	16777216	65536	256	1	增长率
所需步数	68	123	230	460	2.00
计算时间 (s)	0.0014	0.00241	0.00478	0.00766	

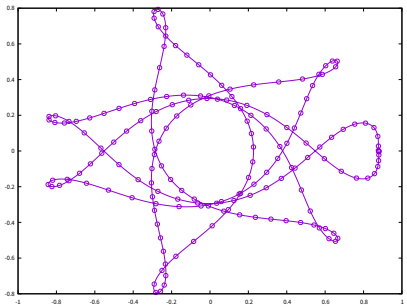
轨道二的结果如下表。

Tolerance($\times 10^{-13}$)	16777216	65536	256	1	增长率
所需步数	107	193	361	723	2.00
计算时间 (s)	0.00234	0.00487	0.00638	0.0121	

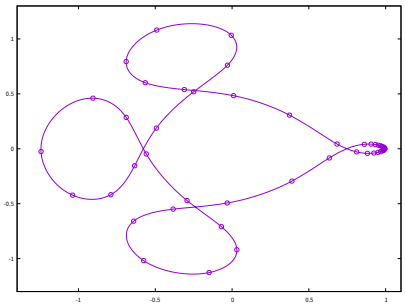
用 DOPRI5 与 DOPRI8 绘制图像，并将求解结点标出，如下。其中 Tolerance 在轨道一中设为 10^{-4}，在轨道二中设为 5×10^{-6}。



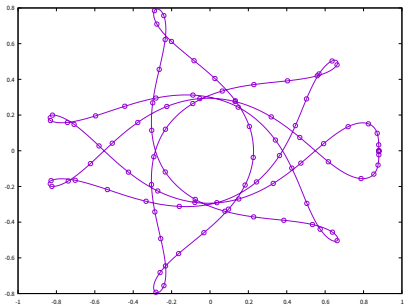
DOPRI5，$n = 64$



DOPRI5，$n = 171$

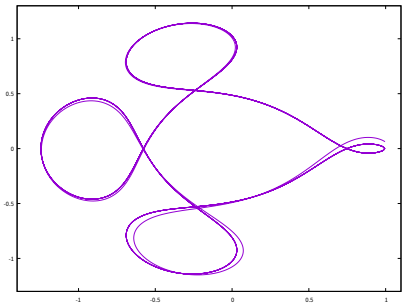


DOPRI8，$n = 47$

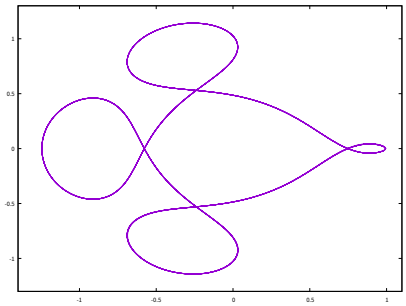


DOPRI8，$n = 95$

我们来比较两种方法在长时间尺度下的稳定性。将轨道一的计算周期由原来的 T 增加至 $5T$，Tolerance 设为 2×10^{-15}，结果绘图如下。



DOPRI5，$n = 40489$，0.292 秒



DOPRI8，$n = 3778$，0.0642 秒

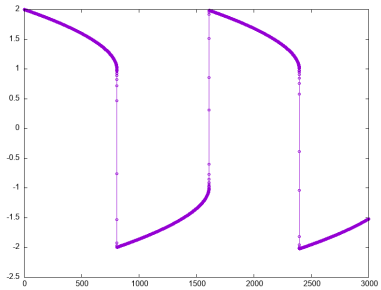
可以看到，DOPRI8 的步数要少得多，且 DOPRI5 计算的轨道在第五个周期发生肉眼可见的偏离，DOPRI8 计算的轨道能够稳定五个周期。

2.5.3 刚性方程测试

用 Van der Pol 方程 (1.2) 测试 Dormand-Prince 方法，在不同的 Tolerance 下求解，DOPRI5 所需步数如下表。

Tolerance	10^{-2}	10^{-3}	10^{-4}
所需步数	1689062	1685911	1684137
计算时间 (s)	7.67	7.99	7.64

可以看到 DOPRI5 使用的步数非常多，而且 Tolerance 对步数的多少影响不大，这是由方程 (1.2) 的性质决定的。取 Tolerance 为 10^{-2} 时的结果绘制图像，并将求解的结点标出，如下图。



我们还测试了 DOPRI8 方法，发现**使用 DOPRI8 方法无法求解！**

2.5.4 结论

Dormand-Prince 方法在非刚性方程中的表现非常好，所需的结点少，且求解速度快。其中 DOPRI5 具有 5 阶精度，DOPRI8 具有 8 阶精度。当一步误差以接近机器精度的 ε 作为 Tolerance 时，累积误差与步数成正相关。而作为最流行的高阶显式数值积分方法，DOPRI8 的高阶收敛性能够让步数更少，使得数值积分保持更长时间的准确性。

另外，Van der Pol 方程的测试表明 Dormand-Prince 方法不适用于刚性方程。

2.6 Fehlberg 方法

2.6.1 非刚性方程测试

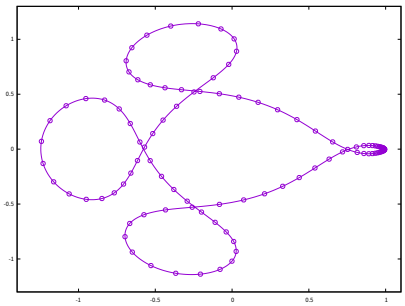
使用限制性三体问题作为样例测试 FB4(5) 方法。Tolerance 从 3.2768×10^{-9} 开始，每次减小至原来的 $\frac{1}{32}$，观察 FB4(5) 的步数需要增加至多少。若步数的增长率接近 2，则可以验证 FB4(5) 方法具有 5 阶收敛性。轨道一的结果如下表。

Tolerance($\times 10^{-13}$)	32768	1024	32	1	增长率
所需步数	513	1027	2053	4106	2.00
计算时间 (s)	0.0035	0.00676	0.0129	0.0231	

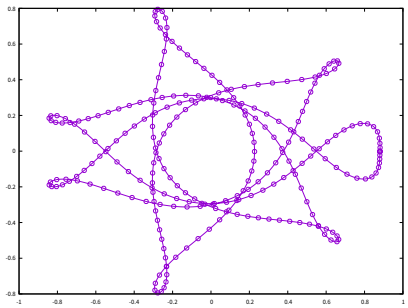
轨道二的结果如下表。

Tolerance($\times 10^{-13}$)	32768	1024	32	1	增长率
所需步数	794	1587	3172	6343	2.00
计算时间 (s)	0.00568	0.01	0.0203	0.0764	

用 FB4(5) 绘制图像，将求解结点标出，如下。其中 Tolerance 在轨道一中设为 10^{-5}，在轨道二中设为 10^{-5}。



FB4(5)，$n = 117$



FB4(5)，$n = 253$

2.6.2 刚性方程测试

用 Van der Pol 方程 (1.2) 测试 Fehlberg 方法，在不同的 Tolerance 下求解，FB4(5) 所需步数如下表。

Tolerance	10^{-2}	10^{-3}	10^{-4}
所需步数	1804746	1812146	1806674
计算时间 (s)	7.74	7.76	6.28

可以看到 FB4(5) 使用的步数非常多，比 DOPRI5 所需的点数更多，而且 Tolerance 对步数的多少影响不大。

2.6.3 结论

Fehlberg 方法在非刚性方程中的表现非常好，具有 5 阶精度。但是各方面性能均不如 DOPRI5。

另外，Van der Pol 方程的测试表明 Fehlberg 方法不适用于刚性方程。

2.7 自适应 Gauss-Legendre 方法

2.7.1 方法简介与实现

实现自适应的 Runge-Kutta 方法需要有一步误差的估计，通常有两种途径，一种是使用 Embedded RK 方法的思路，另一种是使用 Richardson 外插法。在我们初始版本的求解器中，采用了 Rang 在文章 [8] 中提到的方法，考虑范德蒙矩阵：

$$V = \begin{bmatrix} 1 & 1 & \cdots & 1 \\ c_1 & c_2 & \cdots & c_s \\ \vdots & \vdots & \ddots & \vdots \\ c_1^{s-1} & c_2^{s-1} & \cdots & c_s^{s-1} \end{bmatrix}.$$

令 $\hat{\mathbf{e}} = \left(1, \frac{1}{2}, ..., \frac{1}{s-1}, 0\right)^T$，由 $V\hat{\mathbf{b}} = \hat{\mathbf{e}}$ 解出 $\hat{\mathbf{b}}$，从而构建一个 Embedded RK 方法。

然而，经过测试，上述方法的表现非常差。这是因为这样构建的用于一步误差估计的内嵌方法只有 $s-1$ 阶精度，而 GLRKs 本身具有 $2s$ 阶精度，二者阶数相差悬殊，因此一步误差的估计不具有可信度。最终我们采取了 Richardson 外插法来做误差估计，即：

1. $\mathbf{U}^{n+1} = \mathbf{\Phi}(\mathbf{U}^n, t_n, k)$.
2. $\hat{\mathbf{U}}^{\text{mid}} = \mathbf{\Phi}(\mathbf{U}^n, t_n, \frac{k}{2})$.
3. $\hat{\mathbf{U}}^{n+1} = \mathbf{\Phi}(\mathbf{U}^{\text{mid}}, t_n + \frac{k}{2}, \frac{k}{2})$.
4. $\mathbf{E} = \mathbf{U}^{n+1} - \hat{\mathbf{U}}^{n+1}$.

其中 $\mathbf{\Phi}(\mathbf{U}^n, t_n, k)$ 表示用 GLRKs 在 $(\mathbf{U}^n, t_n)$ 处以 k 为步长计算一步的结果。基于这个一步误差估计，再基于讲义 Formula 11.235 中的步长控制方法，我们即可实现 A-GLRK 方法。虽然一步计算的耗时变为了 GLRK 的三倍，但测试结果证明了这样是值得的。

2.7.2 非刚性方程测试

使用限制性三体问题轨道一作为样例测试 A-GLRK 方法，$s = 1$ 时，让 Tolerance 每次下降到原来的 $\frac{1}{8}$，结果如下表。

Tolerance($\times 10^{-12}$)	64	16	4	1	增长率
所需步数	20009	40020	80042	160083	2.00
计算时间 (s)	0.171	0.301	0.553	1.16	

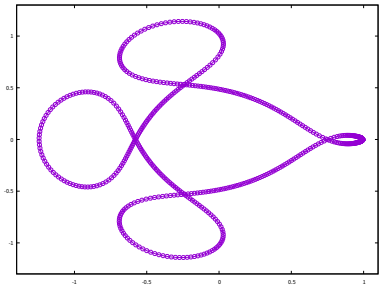
$s=2$ 时，让 Tolerance 每次下降到原来的 $\frac{1}{32}$，结果如下表。

Tolerance($\times 10^{-13}$)	32768	1024	32	1	增长率
所需步数	687	1374	2747	5495	2.00
计算时间 (s)	0.0203	0.0353	0.0629	0.102	

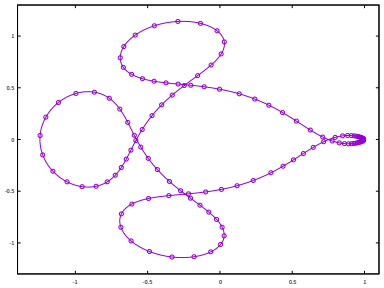
$s=3$ 时，让 Tolerance 每次下降到原来的 $\frac{1}{128}$，结果如下表。

Tolerance($\times 10^{-13}$)	2097152	16384	128	1	增长率
所需步数	112	208	415	831	2.00
计算时间 (s)	0.0112	0.0121	0.0208	0.0363	

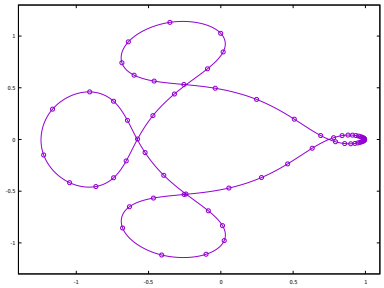
取 Tolerance 为 10^{-4}，用 A-GLRKs ($s=1,2,3$) 方法求解，并将解绘制图像如下。



$s=1$，$n=567$

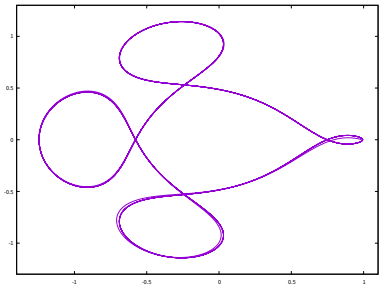


$s=2$，$n=127$

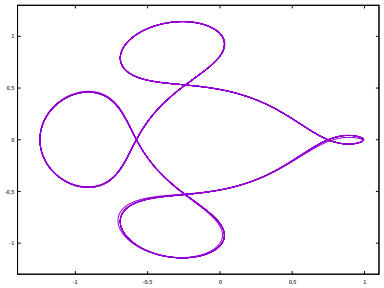


$s=3$，$n=66$

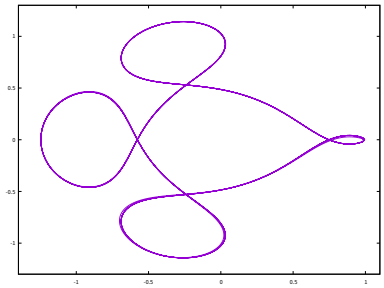
我们取 $s=3,4,5$，将 Tolerance 设为 2×10^{-15}，并将轨道计算 5 个周期，结果如下。



$s=3$，$n=8837$，0.339 秒



$s=4$，$n=2515$，0.177 秒



$s=5$，$n=1155$，0.141 秒

可以看到，$s\geq 4$ 时，使用的点数比 DOPRI8 要少得多，但是计算时间却比它的两倍还多。此外，用 A-GLRK 计算出的轨道都在第五个周期都发生了肉眼可见的偏离。

2.7.3 刚性方程测试

用 Van der Pol 方程 (1.2) 测试 A-GLRK 方法，在不同的 Tolerance 下求解，A-GLRK1 所需步数如下表。

Tolerance	10^{-4}	10^{-6}	10^{-8}
所需步数	918	4544	23675
计算时间 (s)	0.0191	0.0945	0.344

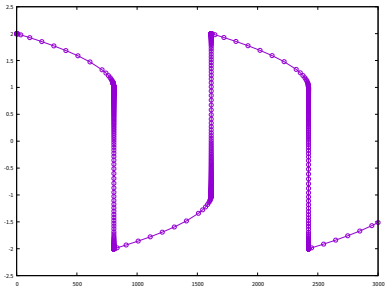
A-GLRK2 所需步数如下表。

Tolerance	10^{-4}	10^{-6}	10^{-8}
所需步数	269	605	1933
计算时间 (s)	0.0862	0.0837	0.175

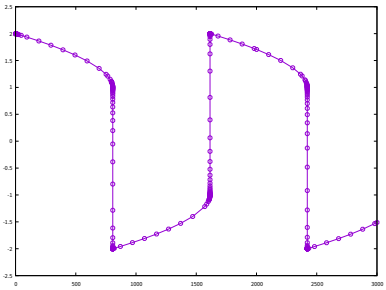
A-GLRK3 所需步数如下表。

Tolerance	10^{-4}	10^{-6}	10^{-8}
所需步数	207	280	530
计算时间 (s)	0.331	0.138	0.12

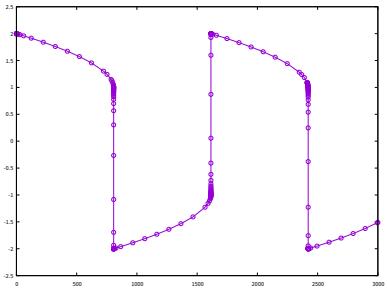
将 Tolerance 取 10^{-4} 时的计算结果绘制图像如下。



$s=1$，$n=918$



$s=2$，$n=269$



$s=3$，$n=207$

2.7.4 结论

在以非刚性方程为例的收敛性测试中，A-GLRKs 方法表现出了 $2s+1$ 阶的收敛率，这是因为采用了 Richardson 外插法作为一步误差估计，从而使得步长控制下的一步误差升了一阶。

与原始的 GLRK 方法对比，A-GLRK 方法优势明显，但在非刚性方程上的表现不如 DOPRI8 方法。

A-GLRK 方法的最大优势在于刚性方程的求解。此外，Dormand-Prince 方法的 Butcher 表需要繁琐的人工计算，但 A-GLRK 方法的 Butcher 表可以自动计算，这让我们能轻松构造高阶自适应数值积分方法。

2.8 自适应 ESDIRK 方法

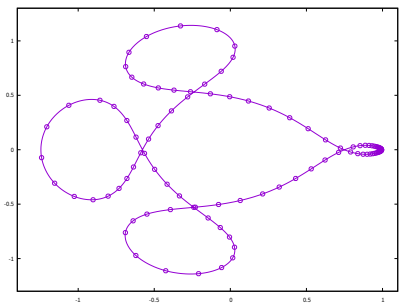
与 A-GLRK 一样，用 Richardson 外插法作为误差估计，来实现自适应 ESDIRK.

2.8.1 非刚性方程测试

使用限制性三体问题轨道一测试 A-ESDIRK 方法，让 Tolerance 每次下降到原来的 $\frac{1}{32}$，结果如下表。

Tolerance($\times 10^{-13}$)	32768	1024	32	1	增长率
所需步数	671	1343	2686	5371	2.00
计算时间 (s)	0.0388	0.0615	0.112	0.212	

设 Tolerance 为 5×10^{-5}，绘制图像如下。



A-ESDIRK，$n=104$

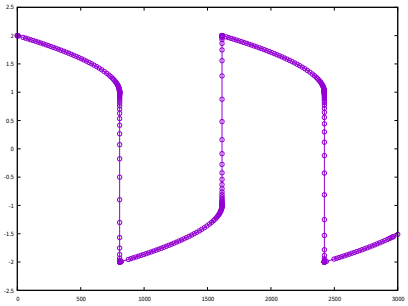
2.8.2 刚性方程测试

用 Van der Pol 方程测试 A-ESDIRK 对刚性方程的求解能力。在不同的 Tolerance 下，需要的步数与计算时间如下表。

Tolerance	10^{-4}	10^{-6}	10^{-8}	10^{-10}
所需步数	376	638	1273	3382
计算时间 (s)	0.375	0.296	0.161	0.254

注意到步数多时计算时间反而少，这与我们隐式方法的迭代实现有关（见 2.2.1 节）。步数少的时候，解的误差比较大，从而隐式方法的第一种迭代不收敛，不得不调用牛顿迭代求解器，后者的速度更慢。而步数多的时候，解的误差小，隐式方法的第一种迭代能够收敛。

将 Tolerance 为 10^{-4} 时的求解结果绘制图像如下。



A-ESDIRK，$n = 376$

2.8.3 结论

在非刚性方程的测试中，A-ESDIRK 表现出了 5 阶收敛性，这是因为采用了 Richardson 外插法作为一步误差估计。它在限制性三体问题上的表现不如 Dormand-Prince 方法。

在 Van der Pol 方程的测试中，A-ESDIRK 的表现比与它同阶的 A-GLRK2 更好，比更高阶的 A-GLRK3 更差，没有充分体现出具有 L 稳定性的优势。我们将在第 10 节进行一个补充测试。

2.9 自适应 Radau-IIA 方法

与 A-GLRK 一样，用 Richardson 外插法作为误差估计，来实现自适应 Radau-IIA.

2.9.1 非刚性方程测试

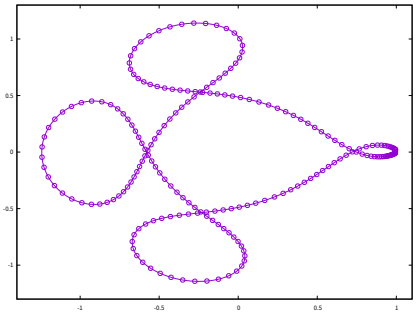
使用限制性三体问题轨道一作为样例测试 A-RIIARK 方法，$s = 2$ 时，让 Tolerance 每次下降到原来的 $\frac{1}{16}$，结果如下表。

Tolerance($\times 10^{-13}$)	4096	256	16	1	增长率
所需步数	3483	6967	13934	27868	2.00
计算时间 (s)	0.0725	0.132	0.244	0.43	

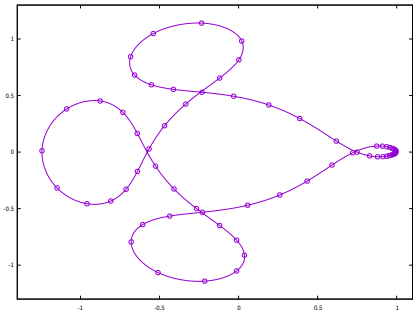
$s = 3$ 时，让 Tolerance 每次下降到原来的 $\frac{1}{64}$，结果如下表。

Tolerance($\times 10^{-13}$)	32768	1024	32	1	增长率
所需步数	687	1374	2747	5495	2.00
计算时间 (s)	0.0203	0.0353	0.0629	0.102	

将 Tolerance 设置到肉眼不可分辨，计算结果绘制图像如下。

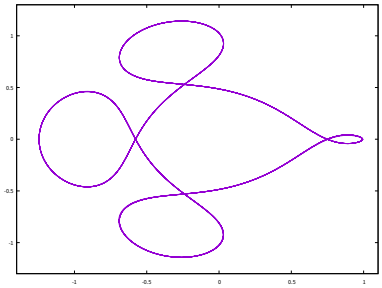


$s=2$，$\varepsilon=2\times10^{-5}$，$n=221$

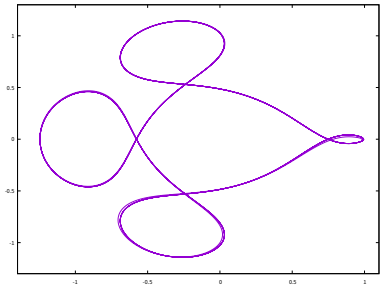


$s=3$，$\varepsilon=10^{-4}$，$n=66$

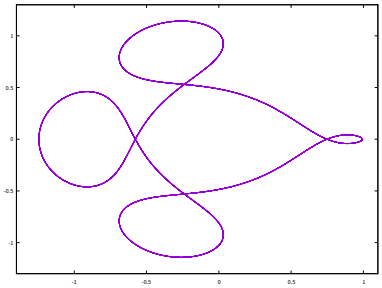
下面来测试 A-RIIARK 在长时间尺度下的求解准确性，将积分区间由 $[0,T]$ 变为 $[0,5T]$，取 $s=4,5,6$，Tolerance 设为 2×10^{-15}，分别绘制轨道如下。



$s=4$，$n=4190$，0.252 秒



$s=5$，$n=2109$，0.227 秒



$s=6$，$n=859$，0.216 秒

当 $s=6$ 时，轨道没有发生肉眼可见的偏离。

2.9.2 刚性方程测试

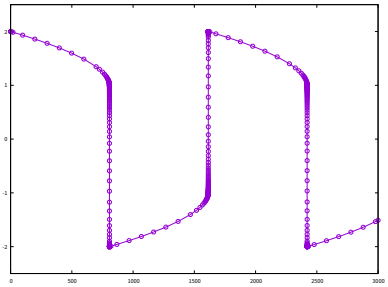
用 Van der Pol 方程 (1.2) 测试 A-RIIARK 方法，在不同的 Tolerance 下求解，A-RIIARK2 所需步数如下表。

Tolerance	10^{-4}	10^{-6}	10^{-8}
所需步数	421	1144	3776
计算时间 (s)	0.115	0.0491	0.111

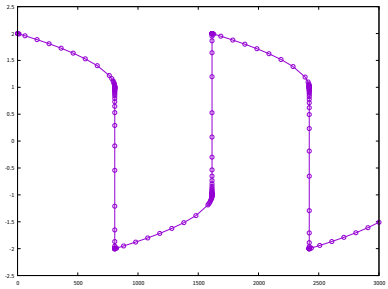
A-RIIARK3 所需步数如下表。

Tolerance	10^{-4}	10^{-6}	10^{-8}
所需步数	200	322	642
计算时间 (s)	0.283	0.112	0.0944

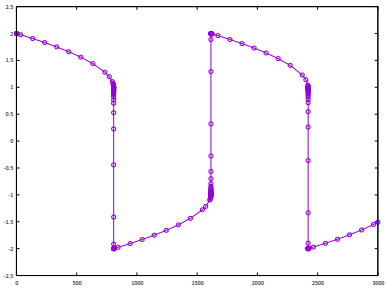
将 Tolerance 取 10^{-4} 时的计算结果绘制图像如下（我们限制了步长最大为 300）。



$s=2$，$n=421$



$s=3$，$n=200$



$s=4$，$n=170$

2.9.3 结论

在以非刚性方程为例的收敛性测试中，A-RIIARKs 方法表现出了 $2s$ 阶的收敛率，这是因为采用了 Richardson 外插法作为一步误差估计，从而使得步长控制下的一步误差升了一阶。

与原始的 RIIARK 方法对比，A-RIIARK 方法优势明显，高阶的 A-RIIARK 在长时间尺度下的稳定度很好，但在非刚性方程上的求解速度不如 DOPRI8 方法。

A-RIIARK 最大的优势在于刚性方程的求解，它在刚性方程上的求解无论是速度还是效果都比 A-ESDIRK 与 A-GLRK 要好。下面这个例子将更好地反映这个问题。

2.10 对比测试：隐式自适应方法的刚性稳定性

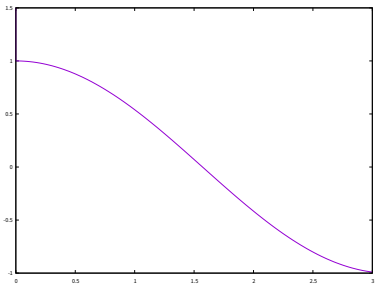
前文的 Van der Pol 方程测试并没有显示出 L 稳定性的作用，因此我们做一组补充测试，目的是凸显 L 稳定性的重要性。考虑讲义 Example 11.144 中的方程：

$$u'(t) = \lambda(u - \cos t) - \sin t, \quad u(0) = \eta. \tag{2.7}$$

方程的真解为：

$$u(t) = e^{\lambda t}(\eta - 1) + \cos t. \tag{2.8}$$

我们取 $\lambda = -10^6, \eta = 1.5$，则它在很短的时间内迅速收敛至余弦曲线，如下图。

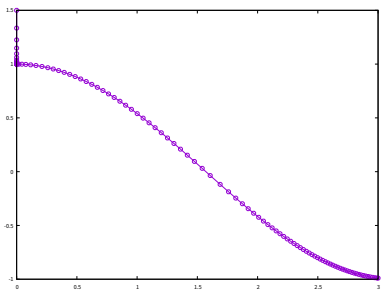


为了方便测试，我们将这个方程也加入了内置函数库。将`"Problem"`参数设为`"Stiff Sample Problem"`即可直接使用。在 $[0, 3]$ 中求解这个方程，在不同的 Tolerance 下，各种隐式自适应方法所需步数与时间如下表。

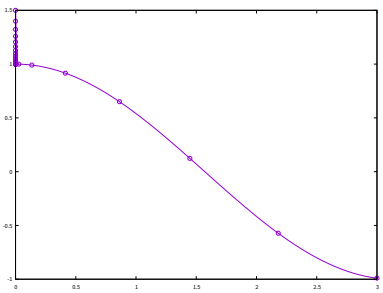
Tolerance	10^{-8}	10^{-10}	10^{-12}	10^{-13}
A-GLRK3	109 步，0.372 秒	367 步，1.36 秒	807 步，2.92 秒	1410 步，5.46 秒
A-RIIARK3	43 步，0.0886 秒	91 步，0.134 秒	227 步，0.388 秒	383 步，0.769 秒
A-ESDIRK	64 步，0.0382 秒	193 步，0.136 秒	948 步，1.16 秒	2497 步，3.7 秒

注意，上面三种方法的阶数分别为 $7, 6, 5$（因为 Richardson 外插法的缘故，可以认为比原始方法高一阶）。但是作为最高阶的 A-GLRK3，在这个方程上居然表现最差，这足以说明 L 稳定性的重要性。

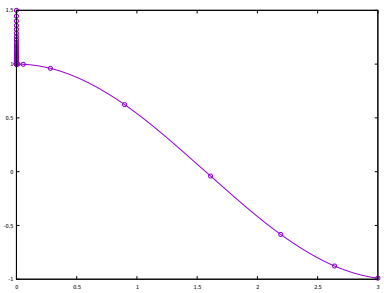
事实上，图像能更好地说明问题。将三种方法在 Tolerance 为 10^{-8} 时的解绘制如下。我们发现，所有方法在一开始取的点都很密集，但具有 L 稳定性的方法在收敛至余弦曲线后迅速增大步长完成计算，不具有 L 稳定性的方法则因为解的振荡而无法增大步长。



A-GLRK3，$n = 109$



A-RIIARK3，$n = 43$

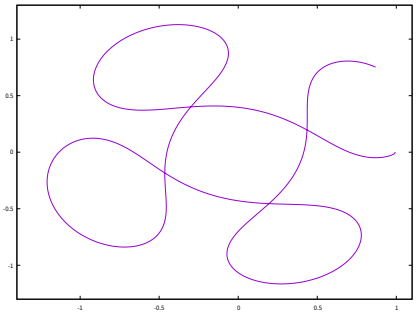


A-ESDIRK，$n = 64$

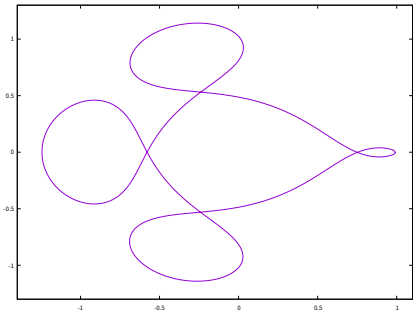
2.11 Adams-Bashforth 方法

2.11.1 初值的敏感性及稳定的计算方法

在限制性三体问题的轨道一中，卫星的起始位置与月球的距离很近，因此受月球引力影响很大，初值稍有一点偏差就会对整个求解过程产生严重的影响。我们现在使用 AB3 求解器，步数设为 50000，前两个初值分别采用向前欧拉法、高阶的 RK 方法计算，结果如下图。



前两个初值由向前欧拉法计算



前两个初值由 RIIARK2 法计算

仅仅是开头的两个初值就对结果产生了如此巨大的影响。因此我们必须采用非常精确的方法计算初值，最好用阶数不低于 p 的方法。在我们的求解器中，一个 p 阶 LMM 方法的前面几个初值，将用 $\lceil \frac{p}{2} \rceil$ 阶段的 Radau-IIA 方法计算。这样可以保证 p 能取得任意大，也通过 Radau-IIA 的 L 稳定性保证了初值的稳定性。

2.11.2 收敛性测试

仍以限制性三体问题做测试。我们使用 ABp 方法，令步长为 $\frac{T}{n}$，取不同的 n，对轨道一测试，将 T 时刻的数值解与初始值比较。$p=1$ 结果如下。

n ($\times 10^7$)		2	4	8	收敛阶
T 时刻误差	u_1	0.000583002	0.000517543	0.000338637	0.61
	u_2	0.00617976	0.00321683	0.001636	0.98
	u_4	0.609395	0.410059	0.236732	0.79
	u_5	0.457729	0.214632	0.093224	1.20
求解时间 (s)		17.6	33.2	69.1	

由于误差太大，收敛阶的估计不可靠。要想得到准确的收敛阶，需要测试更小的步长，但我们的计算机运算能力有限，因此我们放弃这项测试。$p=2$ 结果如下。

n ($\times 10^6$)		2.5	5	10	收敛阶
T 时刻误差	u_1	5.37843e-05	1.32132e-05	3.28959e-06	2.01
	u_2	0.000181596	4.54249e-05	1.13601e-05	2.00
	u_4	0.0298535	0.00740036	0.00184648	2.00
	u_5	0.00778674	0.00202049	0.000509764	1.99
求解时间 (s)		2.05	3.93	8.01	

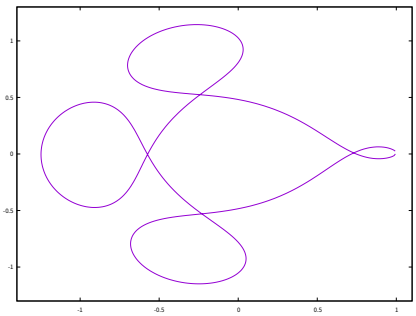
$p=3$ 结果如下。

n ($\times 10^6$)		1	2	4	收敛阶
T 时刻误差	u_1	8.70394e-07	1.11301e-07	1.4072e-08	2.98
	u_2	3.07092e-06	3.921e-07	4.9531e-08	2.98
	u_4	0.000498694	6.36659e-05	8.04244e-06	2.98
	u_5	0.0001352	1.73071e-05	2.18848e-06	2.98
求解时间 (s)		0.791	1.71	3.16	

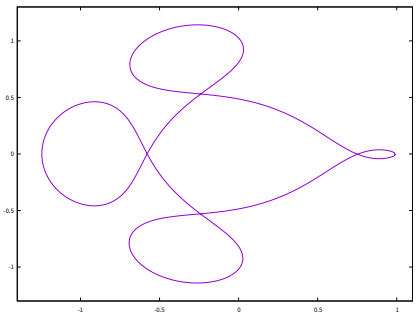
$p=4$ 结果如下。

n $(\times 10^6)$		0.5	1	2	收敛阶
T 时刻误差	u_1	8.05461e-07	5.10134e-08	3.20246e-09	3.99
	u_2	2.70583e-06	1.71126e-07	1.0739e-08	3.99
	u_4	0.000439774	2.78186e-05	1.7458e-06	3.99
	u_5	0.000125495	7.94058e-06	4.98454e-07	3.99
求解时间 (s)		0.414	0.821	1.85	

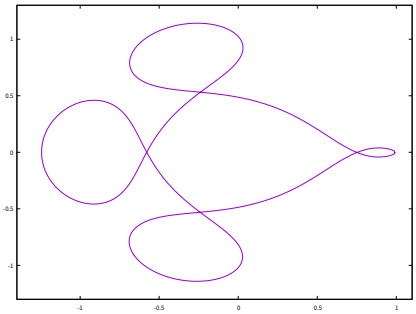
各方法的步数分别取 $4000000, 400000, 50000, 50000$ 绘图如下。



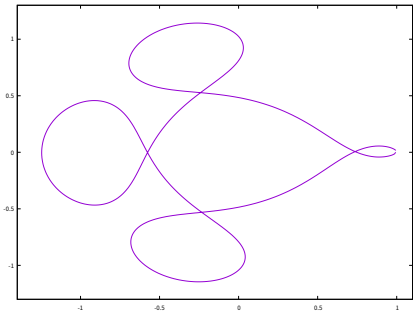
$p=1, n=4000000$



$p=2, n=400000$



$p=3, n=50000$



$p=4, n=50000$

对轨道二测试，这里的收敛阶用 Richardson 外插法估计，$p=1$ 时结果如下。

n $(\times 10^7)$		0.5	1	2	收敛阶
与 $2n$ 步的解在 T 时刻的误差	u_1	0.0055131	0.00264892	0.00129961	1.03
	u_2	0.00521829	0.00259169	0.00129085	1.00
	u_4	0.0254778	0.0117395	0.00564074	1.04
	u_5	0.00988692	0.00512506	0.00259232	0.99
求解时间 (s)		3.53	7.18	15.5	

$p=2$ 时结果如下。

n $(\times 10^6)$		0.5	1	2	收敛阶
与 $2n$ 步的解在 T 时刻的误差	u_1	4.65551e-07	1.16958e-07	2.93113e-08	2.00
	u_2	1.38301e-07	3.40018e-08	8.42821e-09	2.01
	u_4	2.10322e-06	5.28232e-07	1.32365e-07	2.00
	u_5	8.47832e-07	2.13122e-07	5.34268e-08	2.00
求解时间 (s)		0.344	0.672	1.51	

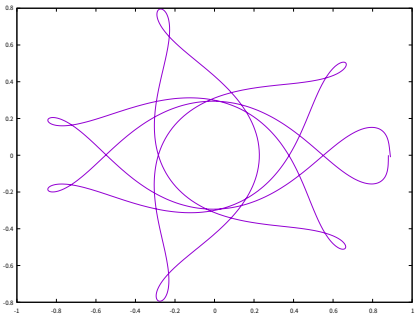
$p=3$ 时结果如下。

n $(\times 10^5)$		1	2	4	收敛阶
与 $2n$ 步的解在 T 时刻的误差	u_1	8.54799e-07	1.06845e-07	1.33545e-08	3.00
	u_2	8.60002e-07	1.07502e-07	1.34368e-08	3.00
	u_4	3.63951e-06	4.54921e-07	5.686e-08	3.00
	u_5	1.74518e-06	2.18139e-07	2.72648e-08	3.00
求解时间 (s)		0.0894	0.182	0.339	

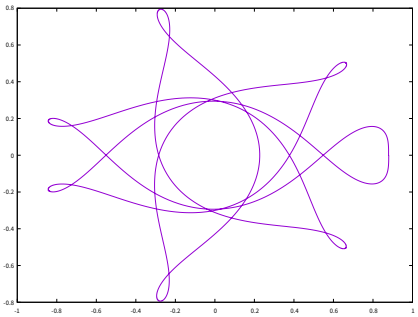
$p=4$ 时结果如下（这里收敛阶超过了 4）。

n ($\times 10^4$)		0.5	1	2	收敛阶
与 $2n$ 步的解在 T 时刻的误差	u_1	2.38152e-05	3.94598e-07	9.64539e-09	5.28
	u_2	3.77951e-05	1.26833e-06	4.49022e-08	4.82
	u_4	9.90411e-05	1.51898e-06	5.15866e-08	4.78
	u_5	5.06315e-05	9.44781e-07	1.05412e-08	6.45
求解时间 (s)		0.0894	0.011	0.0222	

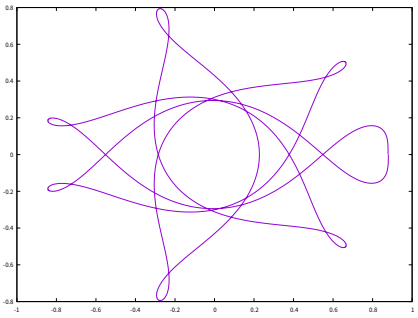
各方法的步数分别取 $5000000, 10000, 8000, 2500$ 绘图如下。



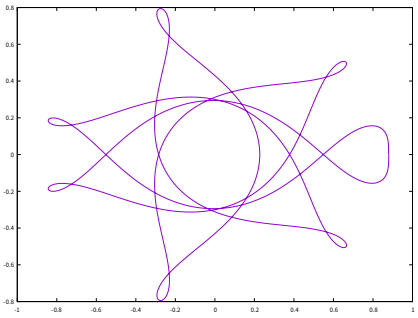
$p=1, n=5000000$



$p=2, n=10000$



$p=3, n=8000$



$p=4, n=2500$

2.11.3 结论

轨道一 $p=1$、轨道二 $p=4$ 的数值测试在收敛阶上出现问题。

ABp 方法具有 p 阶的累积误差，是 p 阶精度的方法，且初值敏感。

2.12 Adams-Moulton 方法

2.12.1 隐式 LMM 的计算方法

与隐式 RK 一样，我们提出两种计算方法。下面是**第一种计算方法**。

1. $\mathbf{U}^{n+s} \leftarrow \mathbf{U}^{n+s-1}$.
2. $\hat{\mathbf{U}}^{n+s} \leftarrow \mathbf{U}^{n+s}$，$\mathbf{U}^{n+s} \leftarrow \mathbf{U}^{n+s-1} + k\sum_{j=1}^{s}\beta_j \mathbf{f}(\mathbf{U}^{n+j}, t_{n+j})$.
3. 若 $||\hat{\mathbf{U}}^{n+s} - \mathbf{U}^{n+s}|| < \varepsilon$，结束，否则返回步骤 2.

第二种计算方法就是牛顿迭代法。当第一种方法计算 40 次不收敛时，就用牛顿迭代法求解。

2.12.2 收敛性测试

仍以限制性三体问题做测试。我们使用 AMp 方法，令步长为 $\frac{T}{n}$，取不同的 n，对轨道一测试，将 T 时刻的数值解与初始值比较。$p=2$ 结果如下。

n $(\times 10^6)$		1	2	4	收敛阶
T 时刻误差	u_1	6.37208e-05	1.6304e-05	4.0996e-06	1.99
	u_2	0.000227146	5.68145e-05	1.42051e-05	2.00
	u_4	0.0363206	0.00919204	0.00230492	2.00
	u_5	0.0107967	0.00259356	0.000641599	2.02
求解时间 (s)		1.47	3.22	5.66	

$p=3$ 结果如下。

n $(\times 10^5)$		4	8	16	收敛阶
T 时刻误差	u_1	1.45118e-06	1.89548e-07	2.42012e-08	2.97
	u_2	5.13822e-06	6.68618e-07	8.52349e-08	2.97
	u_4	0.000833905	0.000108554	1.38394e-05	2.97
	u_5	0.000226146	2.94864e-05	3.76396e-06	2.97
求解时间 (s)		0.676	1.34	2.39	

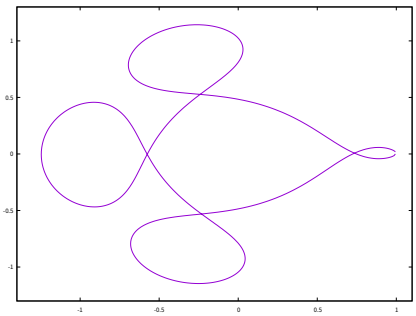
$p=4$ 结果如下。

n $(\times 10^5)$		2	4	8	收敛阶
T 时刻误差	u_1	2.33539e-06	1.49452e-07	9.44687e-09	3.98
	u_2	7.84982e-06	5.01761e-07	3.16867e-08	3.99
	u_4	0.00127668	8.1569e-05	5.15114e-06	3.99
	u_5	0.000362429	2.32574e-05	1.47036e-06	3.98
求解时间 (s)		0.38	0.711	1.49	

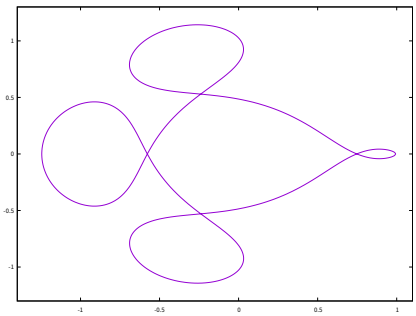
$p=5$ 结果如下（这里收敛阶超过了 5）。

n $(\times 10^5)$		1	2	4	收敛阶
T 时刻误差	u_1	1.44067e-06	2.5908e-08	4.67902e-10	5.79
	u_2	4.54827e-06	7.79218e-08	1.29963e-09	5.90
	u_4	0.000740406	1.27065e-05	2.12444e-07	5.90
	u_5	0.000224524	4.03229e-06	7.28449e-08	5.79
求解时间 (s)		0.203	0.383	0.741	

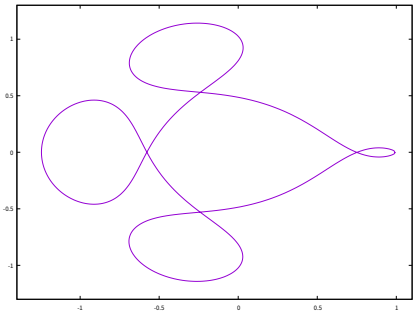
各方法的步数分别取 $100000, 40000, 40000, 10000$ 绘图如下。



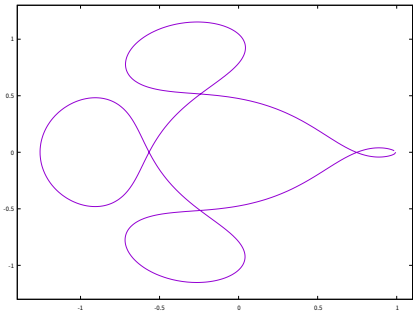
$p=2, n=100000$



$p=2, n=40000$



$p=3, n=40000$



$p=4, n=10000$

对轨道二测试，这里的收敛阶用 Richardson 外插法估计，$p=2$ 时结果如下。

n ($\times 10^5$)		2.5	5	10	收敛阶
与 $2n$ 步的解在 T 时刻的误差	u_1	3.76097e-07	1.25347e-07	2.35032e-08	2.00
	u_2	1.06961e-07	3.56562e-08	6.68756e-09	2.00
	u_4	1.69816e-06	5.65966e-07	1.06122e-07	2.00
	u_5	6.85727e-07	2.28541e-07	4.28523e-08	2.00
求解时间 (s)		0.405	0.819	1.63	

$p=3$ 时结果如下。

n ($\times 10^5$)		0.5	1	2	收敛阶
与 $2n$ 步的解在 T 时刻的误差	u_1	7.59808e-07	1.08523e-07	1.18698e-08	3.00
	u_2	7.64456e-07	1.09208e-07	1.19432e-08	3.00
	u_4	3.2351e-06	4.62059e-07	5.05387e-08	3.00
	u_5	1.55123e-06	2.21566e-07	2.42339e-08	3.00
求解时间 (s)		0.112	0.193	0.368	

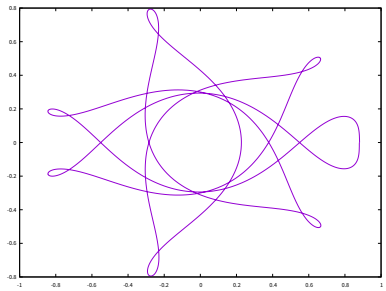
$p=4$ 时结果如下（这里出于未知原因收敛阶不对，增大、减小 n 均不对）。

n ($\times 10^3$)		3	6	12	收敛阶
与 $2n$ 步的解在 T 时刻的误差	u_1	1.07968e-05	1.32633e-07	8.69294e-09	3.73
	u_2	1.89588e-05	6.42769e-07	2.31584e-08	4.79
	u_4	4.45645e-05	4.69373e-07	4.31946e-08	3.15
	u_5	2.3239e-05	3.53269e-07	1.2365e-08	4.73
求解时间 (s)		0.0109	0.0177	0.034	

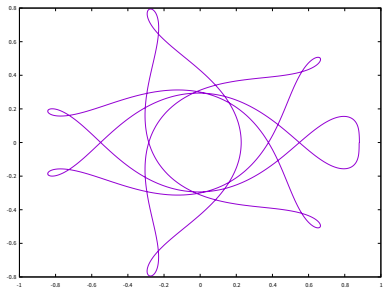
$p=5$ 时结果如下（这里出于未知原因收敛阶又对回来了……）。

n ($\times 10^3$)		3	6	12	收敛阶
与 $2n$ 步的解在 T 时刻的误差	u_1	1.54218e-05	4.86839e-07	1.52429e-08	5.00
	u_2	1.53823e-05	4.87139e-07	1.52768e-08	4.99
	u_4	6.58658e-05	2.07919e-06	6.50954e-08	5.00
	u_5	3.12816e-05	9.87611e-07	3.09256e-08	5.00
求解时间 (s)		0.0105	0.0152	0.0373	

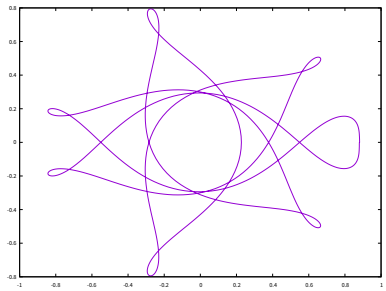
各方法的步数分别取 $5000, 4000, 1200, 1100$ 绘图如下。



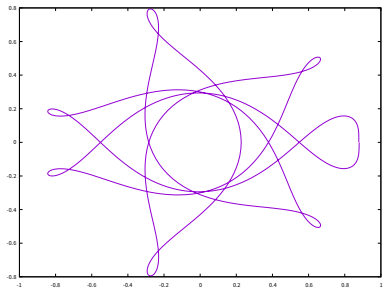
$p=2, n=5000$



$p=3, n=4000$



$p=4, n=1200$



$p=5, n=1100$

2.12.3 结论

轨道一 $p=5$、轨道二 $p=4$ 的数值测试在收敛阶上出现问题。

AMp 方法具有 p 阶的累积误差，是 p 阶精度的方法，且初值敏感。与同阶 AB 方法相比，AM 方法更加精确，但是求解时间相对较长。

2.13 BDF 方法

2.13.1 收敛性测试

仍以限制性三体问题做测试。我们使用 BDFp 方法，令步长为 $\frac{T}{n}$，取不同的 n，对轨道一测试，将 T 时刻的数值解与初始值比较。$p=1$ 结果如下。

n $(\times 10^7)$		3	6	12	收敛阶
T 时刻误差	u_1	0.00203972	0.000792781	0.000341843	1.21
	u_2	0.00425109	0.00219724	0.00110403	0.99
	u_4	0.796148	0.396425	0.190302	1.06
	u_5	0.117455	0.0215009	0.0295195	-
求解时间 (s)		43.3	89	177	

由于误差太大，收敛阶的估计不可靠。要想得到准确的收敛阶，需要测试更小的步长，但我们的计算机运算能力有限，因此我们放弃这项测试。$p=2$ 结果如下。

n $(\times 10^6)$		2	4	8	收敛阶
T 时刻误差	u_1	6.35253e-05	1.62774e-05	4.09461e-06	1.99
	u_2	0.000226494	5.67297e-05	1.41897e-05	2.00
	u_4	0.0362177	0.00917834	0.00230242	2.00
	u_5	0.0107614	0.00258926	0.000640814	2.01
求解时间 (s)		3.61	7.07	12.1	

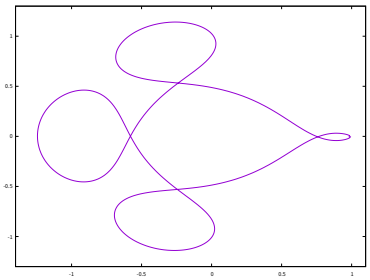
$p=3$ 结果如下。

n $(\times 10^6)$		1	2	4	收敛阶
T 时刻误差	u_1	5.68028e-07	7.34781e-08	9.3957e-09	2.97
	u_2	2.00834e-06	2.59047e-07	3.30353e-08	2.97
	u_4	0.000326018	4.20594e-05	5.36414e-06	2.97
	u_5	8.84106e-05	1.14286e-05	1.46127e-06	2.97
求解时间 (s)		1.71	3.47	6.09	

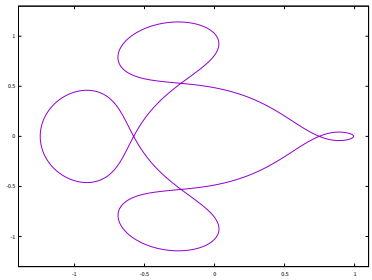
$p=4$ 结果如下。

n $(\times 10^6)$		0.5	1	2	收敛阶
T 时刻误差	u_1	4.60126e-07	2.9195e-08	1.84087e-09	3.99
	u_2	1.5456e-06	9.79572e-08	6.16157e-09	3.99
	u_4	0.000251275	1.59243e-05	1.00172e-06	3.99
	u_5	7.15759e-05	4.54394e-06	2.86524e-07	3.99
求解时间 (s)		0.942	1.74	3.77	

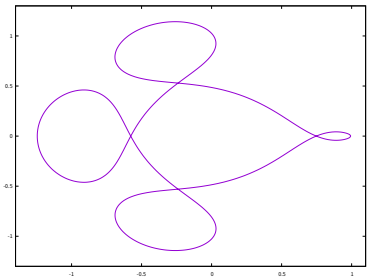
各方法的步数分别取 $8000000, 800000, 80000, 50000$ 绘图如下。



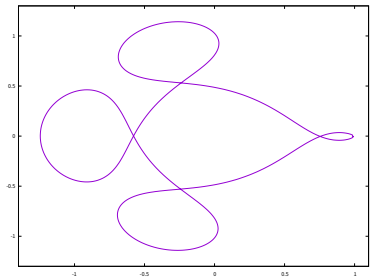
$p=1, n=8000000$



$p=2, n=800000$



$p=3, n=80000$



$p=4, n=50000$

对轨道二测试，这里的收敛阶用 Richardson 外插法估计，$p=1$ 时结果如下。

n $(\times 10^7)$		0.5	1	2	收敛阶
与 $2n$ 步的解在 T 时刻的误差	u_1	0.00477082	0.0024646	0.00125361	0.98
	u_2	0.00504767	0.00254819	0.00127992	0.99
	u_4	0.0186952	0.0100556	0.00522051	0.95
	u_5	0.0105393	0.00527028	0.00262753	1.00
求解时间 (s)		7.65	14.4	32.2	

$p=2$ 时结果如下。

n $(\times 10^5)$		2	4	8	收敛阶
与 $2n$ 步的解在 T 时刻的误差	u_1	2.27916e-06	5.78623e-07	1.45581e-07	1.99
	u_2	7.40341e-07	1.76208e-07	4.31205e-08	2.03
	u_4	1.03094e-05	2.61493e-06	6.57668e-07	1.99
	u_5	4.13995e-06	1.05302e-06	2.65148e-07	1.99
求解时间 (s)		0.361	0.711	1.48	

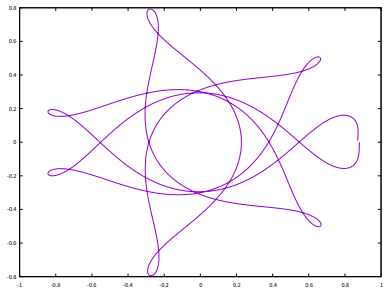
$p=3$ 时结果如下。

n $(\times 10^4)$		2	4	8	收敛阶
与 $2n$ 步的解在 T 时刻的误差	u_1	7.12807e-05	8.90571e-06	1.11306e-06	3.00
	u_2	7.16521e-05	8.95764e-06	1.11977e-06	3.00
	u_4	0.000303654	3.79212e-05	4.73918e-06	3.00
	u_5	0.000145435	1.81804e-05	2.2724e-06	3.00
求解时间 (s)		0.0521	0.101	0.174	

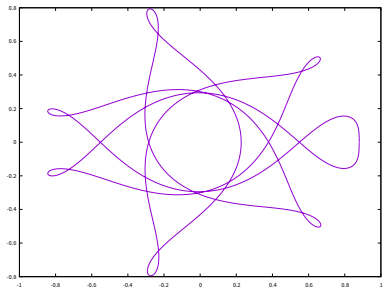
$p=4$ 时结果如下（这里出于未知原因收敛阶不对，增大、减小 n 均不对）。

n $(\times 10^4)$		0.5	1	2	收敛阶
与 $2n$ 步的解在 T 时刻的误差	u_1	1.51285e-05	2.72235e-07	4.09941e-09	6.01
	u_2	2.31412e-05	7.7352e-07	2.71978e-08	4.83
	u_4	6.30961e-05	1.0673e-06	2.347e-08	5.44
	u_5	3.19987e-05	6.34972e-07	3.13921e-09	7.64
求解时间 (s)		0.0174	0.0355	0.0536	

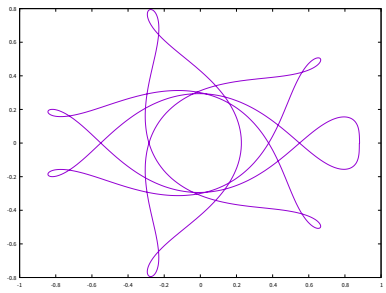
各方法的步数分别取 $5000000, 8000, 8000, 2300$ 绘图如下。



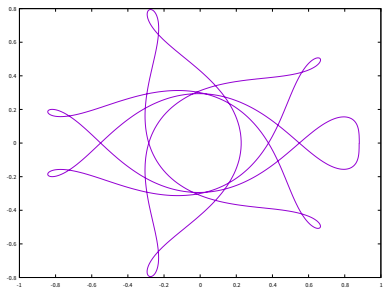
$p=1, n=5000000$



$p=2, n=8000$



$p=3, n=8000$



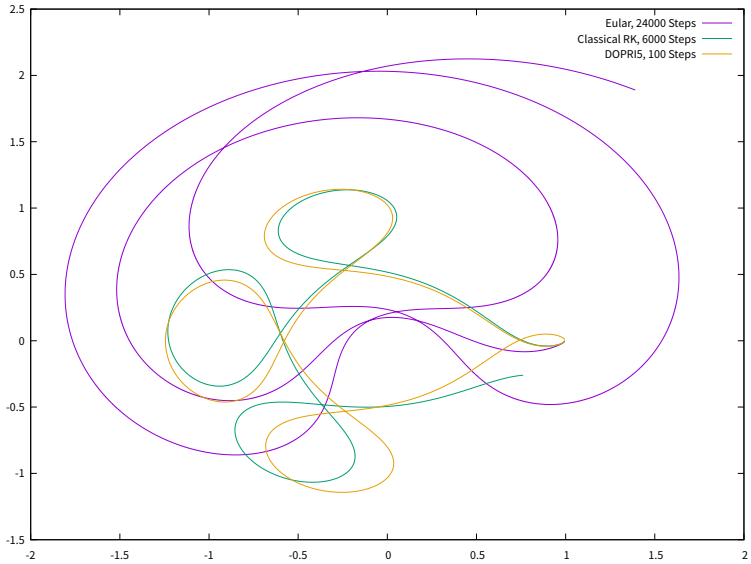
$p=4, n=2300$

2.13.2 结论

BDFp 方法具有 p 阶的累积误差，是 p 阶精度的方法，且初值敏感。

2.14 对比测试：所有方法的求解效果与求解效率

在这一节中，我们以限制性三体问题的轨道一测试所有的方法。首先我们用 24000 步的向前欧拉法、6000 步的经典 RK 法、100 步的 DOPRI5 法（Tolerance 为 10^{-4}，最大步长 0.21）分别求解并绘图如下。



下面我们以计算精度达到 10^{-3} 为目标，比较各方法的耗时。

Method		Tolerance	Steps	CPU Time (s)
Classical RK		-	86000	0.163
ESDIRK		-	66200	0.78
Gauss-Legendre	$s=2$	-	60000	0.311
	$s=3$	-	17200	0.15
	$s=4$	-	7500	0.116
	$s=5$	-	4950	0.112
Radau-IIA	$s=2$	-	340000	1.56
	$s=3$	-	28500	0.246
	$s=4$	-	11700	0.18
	$s=5$	-	5400	0.118
Fehlberg		8.5e-09	424	0.00262
Dormand-Prince	DOPRI5	3.1e-07	190	0.00117
	DOPRI8	2.7e-07	83	0.00156
Adaptive ESDIRK		1.2e-08	518	0.0324
Adaptive Gauss-Legendre	$s=2$	9.0e-08	354	0.0157
	$s=3$	2.4e-08	146	0.0123
	$s=4$	4.9e-08	79	0.0125
	$s=5$	7.0e-08	56	0.102
Adaptive Radau-IIA	$s=2$	3.5e-10	3623	0.0761
	$s=3$	6.7e-09	294	0.0206
	$s=4$	1.8e-08	112	0.0177
	$s=5$	6.0e-08	64	0.0186
Adams-Bashforth	$p=1$	-	-	>5
	$p=2$	-	-	>5
	$p=3$	-	800000	0.602
	$p=4$	-	410000	0.35
	$p=5$	-	156000	0.164
Adams-Moulton	$p=2$	-	-	>5
	$p=3$	-	380000	0.706
	$p=4$	-	213000	0.401
	$p=5$	-	95000	0.203
BDF	$p=1$	-	-	>5
	$p=2$	-	-	>5
	$p=3$	-	682000	1.4
	$p=4$	-	353000	0.806
	$p=5$	-	144000	0.308

可以看到，DOPRI5 在本问题的求解时间上击败了所有方法。但是其它方法也有各自的优势，见文末总结。

第 3 章 总结

在本文中，我们实现了多种方法的数值积分求解器。经过一系列的测试，我们向您给出以下**推荐方法**：

1. 常规的非刚性问题：DOPRI5.
2. 需要长时间高精度求解的非刚性问题：DOPRI8.
3. 常规的刚性问题：A-RIIARK3.
4. 需要长时间高精度求解的刚性问题：A-RIIARK5、A-RIIARK6.

若没有特别的原因，您不必考虑使用其它方法。若上述方法都不能凑效，也许您需要考虑超高精度计算。

上述推荐方法里，大部分是我们在讲义之外添加的新方法。每一个方法的提出都是因为我们碰到了新的难以求解的问题。我们碰到的第一个问题是限制性三体问题的轨道难以长时间计算，这催生了我们寻找高阶 Dormand-Prince 方法的想法，很幸运，Prince 与 Dormand 早就构建了一个 13 阶段 8(7) 阶的显式方法。

在我们碰到第二个问题之前，发生了一段小插曲。因为 Gauss-Legendre 方法的 Butcher 表构造需要求多项式的根，我们采用了数值代数中求矩阵特征根的隐式双重步 QR 迭代法，然而发现 QR 迭代陷入循环。这并不是代码原因，而是 QR 迭代本身的原因，这里我们就不说了。通过与仲杏慧老师的交流，我们最终解决了这个问题，但仲老师也向我们指出，GLRK 方法不具有“非线性稳定性”。我猜测仲老师说的“非线性稳定性”应该是指 L 稳定性。

当时我们测试讲义上的方程 (11.95)，所有的 RK 方法中只有 ESDIRK 能够顺利求解。而测试 Van der Pol 方程后发现，我们已经实现的所有方法都不能很好地求解！这催生了我们实现自适应步长的 ESDIRK 方法的想法。

在这之后，我们发现 A-ESDIRK 的求解速度和效果没有想象中的好，而且不具有长时间求解的能力。我们还想要更高阶且 L 稳定的隐式方法。于时我们再次查阅文献，找到了 Radau-IIA 方法，并且将它的自适应版本也实现。在最终版本的求解器中，自适应 Radau-IIA 击败了所有的隐式方法，成为了我们的推荐方法。

另外，因为三种 LMM 方法的表现都很一般，没有提起我们的兴趣，且 LMM 难以实现步长控制，不方便改造成自适应方法，因此我们不对它们做稳定性测试，也不推荐您使用。

最后要指出本文实验中的不足。我们有一些方法在测试中表现出了不符合理论阶数的收敛阶：

1. 在轨道一的测试中，1 阶 Adams-Bashforth 与 1 阶 Adams-Moulton 都因误差太大而无法准确测试收敛阶；
2. 在轨道一中，5 阶 Adams-Moulton 表现出接近 6 阶的收敛阶；
3. 在轨道二中，经典 RK、ESDIRK、4 阶 Adams-Bashforth、4 阶 BDF 均表现出超过 4 阶的收敛阶，4 阶 Adams-Moulton 在轨道二上收敛阶在 3 到 6 之间波动极大。

一方面是因为收敛阶的测试必须要在步长很小时才会准确，但随着步长变小，机器精度逐渐成为误差的主要来源，并随步数累积，我们无法控制。另一方面是未知原因。。。。幸运的是除了 4 阶 Adams-Moulton 在轨道二上掉阶，其余测试都没有出现掉阶问题，而且每个方法总有一条轨道上的收敛阶是对的。我们猜测，要想测出更准确的收敛阶数，可能要使用高精度浮点运算。我们还有一个更为大胆的猜测，**也许轨道二与四阶方法存在某种特殊的“不相容性”**，这一点有待研究。

总之经过一系列测试，我们对显式方法、隐式方法的区别有了非常深刻的理解，也对稳定性的重要性有了深刻的感受。我们期望这个求解器能帮助您求解更多的常微分方程，若您有我们的求解器无法求解的方程，非常希望您能告诉我们，以便进一步研究。

参考文献

[1] O. Axelsson. “A class of A -stable methods”. In: *Bit Numerical Mathematics* 9.3 (1969), pp. 185–199.

[2] P. E. Gill and W. Murray. “Newton-type methods for unconstrained and linearly constrained optimization”. In: *Mathematical Programming* 7.1 (1974), pp. 311–350.

[3] HAIRER et al. “Solving Ordinary ,Differential Equations II. Stiff and Differential-Algebraic Problems/E. Hairer, G. Wanner, Second Revised Edition with 137 Figures , Vol.: 2”. In: (2002).

[4] E. Hairer, SP Nørsett, and G. Wanner. *Solving ordinary differential equations I : nonstiff problems*. Solving ordinary differential equations I : nonstiff problems, 1987.

[5] C. A. Kennedy and M. H. Carpenter. “Additive Runge-Kutta schemes for convection-diffusion-reaction equations”. In: *Applied Numerical Mathematics* 44.1-2 (2003), pp. 139–181.

[6] J. Nocedal and S. J. Wright. “[Springer Series in Operations Research and Financial Engineering] Numerical Optimization ||”. In: 10.1007/b98874 (1999).

[7] P.J. Prince and J.R. Dormand. “High order embedded Runge-Kutta formulae”. In: *Journal of Computational and Applied Mathematics* 7.1 (1981), pp. 67–75. ISSN: 0377-0427. DOI: https://doi.org/10.1016/0771-050X(81)90010-3. URL: https://www.sciencedirect.com/science/article/pii/0771050X81900103.

[8] Joachim Rang. *Apdative Timestep Control for Fully Implicit Runge-Kutta Methods of Higher Order*. Univ.-Bibl., 2014.

[9] XiaoMing et al. “More than six hundred new families of Newtonian periodic planar collisionless three-body orbits”. In: *Science China(Physics,Mechanics and Astronomy)* 12.v.60 (2017), pp. 68–74.

[10] 叶兴德. 数值分析基础. 数值分析基础, 2008.